Module 3: Catalytic Design

CONTENTS PAGE

01	code/historical/code/s0_env/00_manifest.py	2
02	code/historical/code/s0_env/af_utils.py	4
03	code/historical/code/s1_structure/01_qc.py	7
04	code/historical/code/s1_structure/02_contacts.py	11
05	code/historical/code/s1_structure/03_summary.py	14
06	code/historical/code/s1_structure/04_interactions.py	18
07	code/historical/code/s4_contacts/10_contact_decomposition.py	23
08	code/historical/code/s5_cp/15_cp_dcp_analysis.py	27
09	code/historical/code/s5_cp/19_mut_cp.py	30
10	code/historical/code/s5_cp/52_mutant_baseline_dcp.py	33
11	code/historical/code/s7_wetlab/09_experimental_validation.py	38
12	code/historical/code/s7_wetlab/30_GCG_vs_off_WT.py	41
13	code/historical/code/s7_wetlab/35_dry_vs_wet_ranking.py	46
14	code/historical/code/s7_wetlab/51_metric_reevaluation.py	54
15	code/historical/code/s8_offtarget/58_targeting_ratio_rerank.py	63
16	scripts/audit_science.py	69
17	scripts/compare_esm2.py	73
18	requirements-audit.txt	77
19	code/historical/requirements.txt	78





code/historical/code/s0_env/00_manifest.py

```python
# -*- coding: utf-8 -*-
"""
阶段 0 —— 清单归一化
扫描 3 个 RNA 文件夹，生成 manifest.csv，统一 seed 与 rank，并做完整性断言。

产出：results/manifest.csv
列：rna, seed, rank, path
"""
import os
import csv
import glob
import re

import af_utils as U

OUT_DIR = "results"
os.makedirs(OUT_DIR, exist_ok=True)

rows = []
problems = []

for rd in U.RNA_DIRS:
    # 只把包含 *_job_request.json 的子目录当作 seed 目录
    seed_dirs = [
        d
        for d in glob.glob(os.path.join(rd, "*/"))
        if glob.glob(os.path.join(d, "*_job_request.json"))
    ]

    for sd in sorted(seed_dirs):
        job = glob.glob(os.path.join(sd, "*_job_request.json"))[0]
        seed, rna, protein = U.read_job(job)

        models = sorted(glob.glob(os.path.join(sd, "*_model_[0-4].cif")))
        ranks = []
        for m in models:
            m2 = re.search(r"_model_(\d)\.cif$", m)
            rank = int(m2.group(1))
            ranks.append(rank)
            rows.append([rna, seed, rank, os.path.abspath(m)])

        if len(models) != 5 or sorted(ranks) != [0, 1, 2, 3, 4]:
            problems.append((rd, seed, len(models)))

# 写出 manifest
with open(
    os.path.join(OUT_DIR, "manifest.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.writer(f)
    w.writerow(["rna", "seed", "rank", "path"])
    w.writerows(rows)

# 断言与汇总
print("== manifest 汇总 ==")
from collections import Counter, defaultdict

```





```
by_rna = defaultdict(set)
for rna, seed, rank, path in rows:
    by_rna[rna].add(seed)

all_ok = True
for rna in U.RNA_SEQS:
    n_seed = len(by_rna[rna])
    n_model = sum(1 for r in rows if r[0] == rna)
    flag = "OK" if n_seed == 20 and n_model == 100 else "*** 异常 ***"
    if flag != "OK":
        all_ok = False
    print(f"  {rna}: {n_seed} seeds, {n_model} models  [{flag}]")

print(f"  总计: {len(rows)} 个 model cif")
if problems:
    print("*** 以下 seed 的 model 数量异常：", problems)
    all_ok = False
if all_ok:
    print("  断言通过：每条 RNA 20 个独立 seed × 5 个 model = 100 结构。")
else:
    print("  存在异常，请先检查数据完整性。")
print("写入 results/manifest.csv")
```





code/historical/code/s0_env/af_utils.py

```python
# -*- coding: utf-8 -*-
"""
AF3 cif 质控 + 接触分析 共享工具（纯标准库，无第三方依赖）。

数据约定（已对照实际文件核实）：
- 蛋白链 chain A（asym_id "A"），199 残基，编号 1..199，与 FASTA 编号 1:1 对应。
- RNA 链  chain B（asym_id "B"），7 个核苷酸，编号 1..7。
- 编辑 C 为 0 位 = resi 4；+1 位 = resi 5（三个 RNA 模型一致）。
- cif 无氢原子，只有 C/N/O/P/S 重原子，故"重原子距离"= 全部原子距离。
- pLDDT 存于 _atom_site.B_iso_or_equiv 列。
"""
import json
import os

# 三个 RNA 文件夹（仅 RNA 序列不同，蛋白序列相同）
RNA_DIRS = ["GUGCGCG_20", "GUGCACG_20", "AUGCAGC_20"]

# 对应的 RNA 序列（manifest / 汇总中作为 RNA 标识，非文件夹名）
RNA_SEQS = ["GUGCGCG", "GUGCACG", "AUGCAGC"]

# +1 位残基编号（RNA 链 chain B）
PLUS1_RESI = 5

# 链标识
PROT_CHAIN = "A"
RNA_CHAIN = "B"

# 嘌呤碱基重原子（G/A 并集，程序按实际存在的原子自动取）
BASE_ATOMS = {"N9", "C8", "N7", "C5", "C6", "N6", "O6", "N1", "C2", "N2", "N3", "C4"}

# 碱基完整性判据：+1 为 G 或 A 时各自应具备的全部碱基重原子
PURINE_BASE = {
    "G": {"N9", "C8", "N7", "C5", "C6", "O6", "N1", "C2", "N2", "N3", "C4"},
    "A": {"N9", "C8", "N7", "C5", "C6", "N6", "N1", "C2", "N3", "C4"},
}

# 关键残基（resi -> 期望 resn）。编号已按 cif/FASTA 1:1 核对。
# 注意：交接文档中的 "R67" 应为 Y67（67 位在序列/结构中均为 Tyr，非 Arg）。
KEY_RESIDUES = {
    29: "HIS",
    30: "LYS",
    32: "TYR",
    57: "ASN",
    58: "GLN",
    59: "ALA",
    60: "LYS",
    67: "TYR",
    68: "GLY",
    69: "ARG",
    70: "HIS",
    72: "GLU",
    99: "TRP",
}

# 接触分析距离阈值（Å）
CUTOFFS = [3.5, 4.0, 5.0]
```





```python

# ------------------------------------------------------------------------------
# JSON / cif 读取
# ------------------------------------------------------------------------------
def read_job(job_json_path):
    """从 *_job_request.json 读取 (seed, rna_sequence, protein_sequence)。

    注意：seed 必须从 job_request 读取，目录名不可靠。
    """
    with open(job_json_path, "r", encoding="utf-8") as f:
        d = json.load(f)[0]
    seed = str(d["modelSeeds"][0])
    rna = d["sequences"][1]["rnaSequence"]["sequence"]
    protein = d["sequences"][0]["proteinChain"]["sequence"]
    return seed, rna, protein


def parse_atom_site(cif_path):
    """解析 cif 的 _atom_site 循环。

    返回 (columns, rows)：
    - columns: 形如 ["atom_site.group_PDB", "atom_site.id", ...] 的列名列表
    - rows:    [{col_name: value}, ...]，value 已去除首尾引号
    """
    cols = []
    rows = []
    started = False
    with open(cif_path, "r", encoding="utf-8") as f:
        for line in f:
            s = line.rstrip("\n")
            if s.startswith("_atom_site."):
                cols.append(s[1:])  # 去掉前导下划线
                started = True
                continue
            if started:
                if s.startswith(("ATOM", "HETATM")):
                    parts = s.split()
                    rec = {}
                    for c, v in zip(cols, parts):
                        rec[c] = v.strip('"')
                    rows.append(rec)
                else:
                    break  # 数据行结束
    return cols, rows


# ------------------------------------------------------------------------------
# 原子筛选与几何
# ------------------------------------------------------------------------------
def atoms_of(rows, chain, resid=None, names=None, resn=None):
    """按 chain / resid / 原子名 / 残基名 过滤原子。"""
    out = []
    for a in rows:
        if a.get("atom_site.label_asym_id") != chain:
            continue
        if resid is not None and int(a["atom_site.label_seq_id"]) != resid:
            continue
        if names is not None and a.get("atom_site.label_atom_id") not in names:
            continue
        if resn is not None and a.get("atom_site.label_comp_id") not in resn:
```





```
            continue
        out.append(a)
    return out


def residues_by_chain(rows, chain):
    """返回 {resi: [atom_dict, ...]}，按链分组。"""
    res = {}
    for a in rows:
        if a.get("atom_site.label_asym_id") != chain:
            continue
        resi = int(a["atom_site.label_seq_id"])
        res.setdefault(resi, []).append(a)
    return res


def coord(a):
    return (
        float(a["atom_site.Cartn_x"]),
        float(a["atom_site.Cartn_y"]),
        float(a["atom_site.Cartn_z"]),
    )


def min_dist(atoms_a, atoms_b):
    """两个原子列表间的最小重原子距离；任一侧为空返回 None。"""
    if not atoms_a or not atoms_b:
        return None
    best = None
    for a in atoms_a:
        xa, ya, za = coord(a)
        for b in atoms_b:
            xb, yb, zb = coord(b)
            dx, dy, dz = xa - xb, ya - yb, za - zb
            d = dx * dx + dy * dy + dz * dz
            if best is None or d < best:
                best = d
    return best**0.5


def atom_plddt(a):
    """返回单个原子的 pLDDT（B-factor 列）。"""
    try:
        return float(a["atom_site.B_iso_or_equiv"])
    except (KeyError, ValueError):
        return None
```





code/historical/code/s1_structure/01_qc.py

```python
# -*- coding: utf-8 -*-
"""
阶段 1 —— 质控（标准档）
对 manifest 中每一个 model cif 逐条做结构完整性与置信度检查。

硬性剔除（HARD_FAIL，接触分析会跳过）：
  - 缺 chain B（RNA 链）
  - chain A 残基数 != 199（蛋白链不完整）
  - +1 位（chain B resi 5）碱基重原子不齐全（无法做碱基接触）

软警告（WARN，仅标记，不剔除）：
  - +1 碱基平均 pLDDT < 60
  - iptm < 0.5
  - has_clash = True

产出：results/qc_report.csv 、results/qc_summary.md
"""
import os
import csv
import glob
import json
import re

import af_utils as U

OUT_DIR = "results"
os.makedirs(OUT_DIR, exist_ok=True)

# 阈值（标准档）
PLDDT_WARN = 60.0
IPTM_WARN = 0.5


def read_summary_confidences(seed_dir, rank):
    """读取对应 rank 的 summary_confidences JSON，返回 dict。"""
    pat = os.path.join(seed_dir, f"*_summary_confidences_{rank}.json")
    hits = glob.glob(pat)
    if not hits:
        return {}
    with open(hits[0], "r", encoding="utf-8") as f:
        return json.load(f)


def mean(xs):
    xs = [x for x in xs if x is not None]
    return sum(xs) / len(xs) if xs else None


def min_of(xs):
    xs = [x for x in xs if x is not None]
    return min(xs) if xs else None


report = []
count = {"total": 0, "hard_fail": 0, "warn": 0, "pass": 0}

```

```
with open(os.path.join(OUT_DIR, "manifest.csv"), encoding="utf-8-sig") as f:
    reader = csv.DictReader(f)
    for row in reader:
        rna, seed, rank, path = row["rna"], row["seed"], int(row["rank"]), row["path"]
        count["total"] += 1

        flags = []
        # 解析结构
        cols, atoms = U.parse_atom_site(path)
        rA = U.residues_by_chain(atoms, U.PROT_CHAIN)
        rB = U.residues_by_chain(atoms, U.RNA_CHAIN)

        # +1 碱基
        plus1_all = U.atoms_of(atoms, U.RNA_CHAIN, U.PLUS1_RESI)
        plus1_resn = plus1_all[0]["atom_site.label_comp_id"] if plus1_all else "?"
        plus1_base = U.atoms_of(atoms, U.RNA_CHAIN, U.PLUS1_RESI, names=U.BASE_ATOMS)
        base_names = set(a["atom_site.label_atom_id"] for a in plus1_base)
        expected = U.PURINE_BASE.get(plus1_resn, set())
        base_complete = base_names >= expected

        # 结构完整性
        chainA_n = len(rA)
        chainB_n = len(rB)
        if chainB_n == 0:
            flags.append("HARD:no_RNA_chainB")
        if chainA_n != 199:
            flags.append("HARD:chainA_res!=199")
        if not base_complete:
            flags.append("HARD:plus1_base_incomplete")

        # 置信度
        conf = read_summary_confidences(os.path.dirname(path), rank)
        iptm = conf.get("iptm")
        ptm = conf.get("ptm")
        ranking = conf.get("ranking_score")
        has_clash = conf.get("has_clash")
        frac_dis = conf.get("fraction_disordered")

        plus1_plddt = [U.atom_plddt(a) for a in plus1_base]
        plus1_plddt_mean = mean(plus1_plddt)
        plus1_plddt_min = min_of(plus1_plddt)

        # 关键残基 CA pLDDT
        key_ca_plddt = []
        for resi, exp in U.KEY_RESIDUES.items():
            ca = U.atoms_of(atoms, U.PROT_CHAIN, resi, names={"CA"})
            if ca:
                key_ca_plddt.append(U.atom_plddt(ca[0]))
        key_ca_mean = mean(key_ca_plddt)

        if plus1_plddt_mean is not None and plus1_plddt_mean < PLDDT_WARN:
            flags.append("WARN:plus1_plddt<%.0f" % PLDDT_WARN)
        if iptm is not None and iptm < IPTM_WARN:
            flags.append("WARN:iptm<%.1f" % IPTM_WARN)
        if has_clash:
            flags.append("WARN:has_clash")

        hard_fail = any(f.startswith("HARD:") for f in flags)
        warn = any(f.startswith("WARN:") for f in flags)
        if hard_fail:
            count["hard_fail"] += 1
```





```
        elif warn:
            count["warn"] += 1
        else:
            count["pass"] += 1

        report.append(
            {
                "rna": rna,
                "seed": seed,
                "rank": rank,
                "chainA_res": chainA_n,
                "chainB_res": chainB_n,
                "plus1_resn": plus1_resn,
                "plus1_base_complete": base_complete,
                "plus1_base_plddt_mean": (
                    round(plus1_plddt_mean, 2) if plus1_plddt_mean is not None else ""
                ),
                "plus1_base_plddt_min": (
                    round(plus1_plddt_min, 2) if plus1_plddt_min is not None else ""
                ),
                "key_ca_plddt_mean": (
                    round(key_ca_mean, 2) if key_ca_mean is not None else ""
                ),
                "iptm": iptm,
                "ptm": ptm,
                "ranking_score": ranking,
                "has_clash": has_clash,
                "fraction_disordered": frac_dis,
                "qc_pass": not hard_fail,
                "qc_flags": ";".join(flags),
            }
        )

# 写出 csv
fieldnames = list(report[0].keys())
with open(
    os.path.join(OUT_DIR, "qc_report.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=fieldnames)
    w.writeheader()
    w.writerows(report)

# 按 rna 汇总
from collections import defaultdict

summ = defaultdict(lambda: {"n": 0, "fail": 0, "warn": 0, "pass": 0})
for r in report:
    s = summ[r["rna"]]
    s["n"] += 1
    if not r["qc_pass"]:
        s["fail"] += 1
    elif r["qc_flags"]:
        s["warn"] += 1
    else:
        s["pass"] += 1

lines = ["# QC 汇总（标准档）", ""]
lines.append(
    f"- 总计 {count['total']} 个 model；硬剔除 {count['hard_fail']}，警告 {count['warn']}，通过 {count
['pass']}。"
)
```





```
lines.append("")
lines.append("| RNA | 总数 | 通过 | 警告 | 硬剔除 |")
lines.append("|---|---|---|---|---|")
for rna in U.RNA_SEQS:
    s = summ[rna]
    lines.append(f"| {rna} | {s['n']} | {s['pass']} | {s['warn']} | {s['fail']} |")

# 列出被硬剔除的模型
failed = [r for r in report if not r["qc_pass"]]
if failed:
    lines.append("")
    lines.append("## 硬剔除模型清单")
    lines.append("")
    for r in failed:
        lines.append(f"- {r['rna']} seed={r['seed']} rank={r['rank']}  [{r['qc_flags']}]")

with open(os.path.join(OUT_DIR, "qc_summary.md"), "w", encoding="utf-8") as f:
    f.write("\n".join(lines) + "\n")

print("== QC 汇总 ==")
print("\n".join(lines))
print("\n写入 results/qc_report.csv, results/qc_summary.md")
```





code/historical/code/s1_structure/02_contacts.py

```python
# -*- coding: utf-8 -*-
"""
阶段 2 —— +1 位接触分析（双口径）

对每个通过质控的模型：
  - 提取 +1 位（chain B resi 5）的「碱基重原子」与「整核苷酸重原子」两套原子；
  - 计算每个蛋白残基（chain A）到这两套原子的最小重原子距离；
  - 按 3.5 / 4.0 / 5.0 Å 三档判定是否接触。

产出：
  results/contacts_long.csv   —— 每模型 × 199 蛋白残基 的完整距离表
  results/key_distances.csv   —— 关键残基子集（便于 G/A 对比）

依赖 results/manifest.csv 与 results/qc_report.csv（先跑 00、01）。
"""
import os
import csv

import af_utils as U

OUT_DIR = "results"
os.makedirs(OUT_DIR, exist_ok=True)

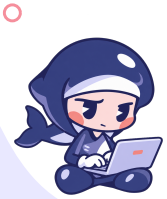

def coord_list(atoms):
    return [U.coord(a) for a in atoms]


def min_dist_to_coords(atom_coords, ligand_coords):
    """原子坐标列表 -> 配体坐标列表 的最小距离。"""
    if not atom_coords or not ligand_coords:
        return None
    best = None
    for xa, ya, za in atom_coords:
        for xb, yb, zb in ligand_coords:
            dx, dy, dz = xa - xb, ya - yb, za - zb
            d = dx * dx + dy * dy + dz * dz
            if best is None or d < best:
                best = d
    return best**0.5


# 读取 manifest 与 qc（跳过硬剔除模型）
manifest = []
with open(os.path.join(OUT_DIR, "manifest.csv"), encoding="utf-8-sig") as f:
    for row in csv.DictReader(f):
        manifest.append(row)

skip = set()
with open(os.path.join(OUT_DIR, "qc_report.csv"), encoding="utf-8-sig") as f:
    for row in csv.DictReader(f):
        if row["qc_pass"] == "False":
            skip.add((row["rna"], row["seed"], int(row["rank"])))

long_rows = []
key_rows = []
```





```

for row in manifest:
    rna, seed = row["rna"], row["seed"]
    rank = int(row["rank"])
    path = row["path"]
    if (rna, seed, rank) in skip:
        continue

    cols, atoms = U.parse_atom_site(path)
    plus1_all = U.atoms_of(atoms, U.RNA_CHAIN, U.PLUS1_RESI)
    plus1_base = [a for a in plus1_all if a["atom_site.label_atom_id"] in U.BASE_ATOMS]

    base_coords = coord_list(plus1_base)
    full_coords = coord_list(plus1_all)

    rA = U.residues_by_chain(atoms, U.PROT_CHAIN)
    for resi in sorted(rA):
        res_atoms = rA[resi]
        resn = res_atoms[0]["atom_site.label_comp_id"]
        rc = coord_list(res_atoms)

        d_base = min_dist_to_coords(rc, base_coords)
        d_full = min_dist_to_coords(rc, full_coords)

        rec = {
            "rna": rna,
            "seed": seed,
            "rank": rank,
            "resi": resi,
            "resn": resn,
            "min_dist_base": ("" if d_base is None else round(d_base, 3)),
            "min_dist_full": ("" if d_full is None else round(d_full, 3)),
        }
        for c in U.CUTOFFS:
            rec[f"contact_base_{c}"] = d_base is not None and d_base <= c
            rec[f"contact_full_{c}"] = d_full is not None and d_full <= c
        long_rows.append(rec)

        if resi in U.KEY_RESIDUES:
            key_rows.append(rec)

# 写 contacts_long.csv
long_fields = [
    "rna",
    "seed",
    "rank",
    "resi",
    "resn",
    "min_dist_base",
    "min_dist_full",
    "contact_base_3.5",
    "contact_base_4.0",
    "contact_base_5.0",
    "contact_full_3.5",
    "contact_full_4.0",
    "contact_full_5.0",
]
with open(
    os.path.join(OUT_DIR, "contacts_long.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=long_fields)
```





```
    w.writeheader()
    w.writerows(long_rows)

# 写 key_distances.csv（沿用相同列）
with open(
    os.path.join(OUT_DIR, "key_distances.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=long_fields)
    w.writeheader()
    w.writerows(key_rows)

print(f"== 接触分析完成 ==")
print(f"  处理 model 数: {len(manifest) - len(skip)}（跳过硬剔除 {len(skip)}）")
print(f"  contacts_long.csv: {len(long_rows)} 行")
print(f"  key_distances.csv: {len(key_rows)} 行")
```





code/historical/code/s1_structure/03_summary.py

```python
# -*- coding: utf-8 -*-
"""
阶段 3 —— 汇总统计与 G/A 对比

读取 results/contacts_long.csv，产出：
  results/contacts_frequency.csv —— 每 (RNA × 蛋白残基) 的接触频率 + 距离中位数
  results/g_a_comparison.csv     —— 同一残基在 G 组 vs A 组的接触频率对比

分组口径：
  GUGCGCG  = "+1=G"（on-target）
  GUGCACG  = "+1=A"（纯 +1 交换，主对照）
  AUGCAGC  = "+1=A"（871 脱靶，侧翼不同，作补充）
  A_pooled = GUGCACG + AUGCAGC
"""
import os
import csv
from collections import defaultdict

import af_utils as U

OUT_DIR = "results"

RNA_GROUPS = {
    "GUGCGCG": "+1=G (on)",
    "GUGCACG": "+1=A (swap)",
    "AUGCAGC": "+1=A (off)",
}
G_RNA = "GUGCGCG"
A_PURE = "GUGCACG"
A_OFF = "AUGCAGC"
A_POOL = [A_PURE, A_OFF]


def median(xs):
    xs = sorted(xs)
    n = len(xs)
    if n == 0:
        return None
    if n % 2:
        return xs[n // 2]
    return (xs[n // 2 - 1] + xs[n // 2]) / 2.0


# 读取
data = defaultdict(
    lambda: defaultdict(
        lambda: {
            "n": 0,
            "n_base5": 0,
            "n_full5": 0,
            "n_base4": 0,
            "n_base3p5": 0,
            "dists_base": [],
            "dists_full": [],
        }
    )
```





```
)
resn_map = {}

with open(os.path.join(OUT_DIR, "contacts_long.csv"), encoding="utf-8-sig") as f:
    for row in csv.DictReader(f):
        rna = row["rna"]
        resi = int(row["resi"])
        resn_map[resi] = row["resn"]
        d = data[rna][resi]
        d["n"] += 1
        if row["contact_base_5.0"] == "True":
            d["n_base5"] += 1
        if row["contact_base_4.0"] == "True":
            d["n_base4"] += 1
        if row["contact_base_3.5"] == "True":
            d["n_base3p5"] += 1
        if row["contact_full_5.0"] == "True":
            d["n_full5"] += 1
        if row["min_dist_base"] != "":
            d["dists_base"].append(float(row["min_dist_base"]))
        if row["min_dist_full"] != "":
            d["dists_full"].append(float(row["min_dist_full"]))

# 1) 频率表
freq_rows = []
for rna in U.RNA_SEQS:
    for resi in sorted(data[rna]):
        d = data[rna][resi]
        n = d["n"]
        freq_rows.append(
            {
                "rna": rna,
                "resi": resi,
                "resn": resn_map[resi],
                "n_model": n,
                "freq_base_3.5": round(d["n_base3p5"] / n, 3),
                "freq_base_4.0": round(d["n_base4"] / n, 3),
                "freq_base_5.0": round(d["n_base5"] / n, 3),
                "freq_full_5.0": round(d["n_full5"] / n, 3),
                "median_min_dist_base": (
                    "" if not d["dists_base"] else round(median(d["dists_base"]), 3)
                ),
                "median_min_dist_full": (
                    "" if not d["dists_full"] else round(median(d["dists_full"]), 3)
                ),
            }
        )

freq_fields = [
    "rna",
    "resi",
    "resn",
    "n_model",
    "freq_base_3.5",
    "freq_base_4.0",
    "freq_base_5.0",
    "freq_full_5.0",
    "median_min_dist_base",
    "median_min_dist_full",
]
with open(
```





```
    os.path.join(OUT_DIR, "contacts_frequency.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=freq_fields)
    w.writeheader()
    w.writerows(freq_rows)


# 2) G/A 对比表
def freq(rna, resi, key):
    d = data[rna].get(resi)
    if not d or d["n"] == 0:
        return 0.0
    return d[key] / d["n"]


all_resi = set()
for rna in U.RNA_SEQS:
    all_resi |= set(data[rna].keys())

comp_rows = []
for resi in sorted(all_resi):
    fG = freq(G_RNA, resi, "n_base5")
    fA_pure = freq(A_PURE, resi, "n_base5")
    fA_off = freq(A_OFF, resi, "n_base5")
    # pooled A：合并两个 A 组的计数
    nA = data[A_PURE].get(resi, {}).get("n_base5", 0) + data[A_OFF].get(resi, {}).get(
        "n_base5", 0
    )
    nA_total = data[A_PURE].get(resi, {}).get("n", 0) + data[A_OFF].get(resi, {}).get(
        "n", 0
    )
    fA_pool = (nA / nA_total) if nA_total else 0.0

    comp_rows.append(
        {
            "resi": resi,
            "resn": resn_map[resi],
            "freq_G": round(fG, 3),
            "freq_A_swap": round(fA_pure, 3),
            "freq_A_off": round(fA_off, 3),
            "freq_A_pooled": round(fA_pool, 3),
            "delta_G_minus_Aswap": round(fG - fA_pure, 3),
            "delta_G_minus_Apooled": round(fG - fA_pool, 3),
        }
    )

comp_fields = [
    "resi",
    "resn",
    "freq_G",
    "freq_A_swap",
    "freq_A_off",
    "freq_A_pooled",
    "delta_G_minus_Aswap",
    "delta_G_minus_Apooled",
]
with open(
    os.path.join(OUT_DIR, "g_a_comparison.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=comp_fields)
    w.writeheader()
```





```
    w.writerows(comp_rows)

# 控制台：打印 +1 碱基接触频率前 20 的残基（按 G 组频率）
print("== +1 碱基(≤5Å)接触频率：按 G 组(GCG)排序的前 20 个蛋白残基 ==")
print(f"{'resi':>5} {'resn':>4} {'G(on)':>7} {'A(swap)':>8} {'A(off)':>8} {'ΔG-A':>7}")
top = sorted(comp_rows, key=lambda r: -r["freq_G"])[:20]
for r in top:
    print(
        f"{r['resi']:>5} {r['resn']:>4} {r['freq_G']:>7} {r['freq_A_swap']:>8} "
        f"{r['freq_A_off']:>8} {r['delta_G_minus_Aswap']:>7}"
    )

print(f"\n== 关键残基 G/A 对比 ==")
print(f"{'resi':>5} {'resn':>4} {'G(on)':>7} {'A(swap)':>8} {'A(off)':>8} {'ΔG-A':>7}")
for r in comp_rows:
    if r["resi"] in U.KEY_RESIDUES:
        print(
            f"{r['resi']:>5} {r['resn']:>4} {r['freq_G']:>7} {r['freq_A_swap']:>8} "
            f"{r['freq_A_off']:>8} {r['delta_G_minus_Aswap']:>7}"
        )

print("\n写入 results/contacts_frequency.csv, results/g_a_comparison.csv")
```





code/historical/code/s1_structure/04_interactions.py

```python
# -*- coding: utf-8 -*-
"""
阶段 4 —— +1 位蛋白-RNA 相互作用分析（PISA 等价：氢键计数 + 界面埋藏面积）

对每个模型的蛋白残基量化其与 +1 位核苷酸的相互作用：
  1) 氢键计数（几何判定：供体-受体重原子距离 <=3.5 Å，无 H 原子近似）
  2) 界面埋藏面积（Shrake-Rupley SASA：残基在「蛋白+RNA」与「仅蛋白」两种
     语境下的 SASA 差 = 被 RNA 埋藏的面积；仅对每 seed 的 rank0 代表结构计算，
     以控制耗时）

产出：
  results/hbonds_per_residue.csv        —— 每 RNA × 残基 的氢键均值/频率（全部 300 模型）
  results/buried_area_per_residue.csv   —— 每 RNA × 残基 的埋藏面积均值（rank0 代表结构）
  results/filtered_residues.csv         —— 再次筛选后的可设计残基清单
"""
import os
import csv
from collections import defaultdict

import af_utils as U

OUT_DIR = "results"

# ---- 氢键供体/受体原子名 ----
PROT_DONOR = {
    "N",
    "NE",
    "NH1",
    "NH2",
    "NZ",
    "ND1",
    "NE2",
    "ND2",
    "NE1",
    "OG",
    "OG1",
    "OH",
}
PROT_ACC = {"O", "OD1", "OD2", "OE1", "OE2", "OG", "OG1", "OH", "ND1", "NE2"}
RNA_DONOR = {"G": {"N1", "N2"}, "A": {"N6"}}
RNA_ACC = {"G": {"N3", "N7", "O6"}, "A": {"N1", "N3", "N7"}}

HB_DIST = 3.5

# 绝对禁区（不可突变）
FORBIDDEN = {57, 70, 72, 99, 102, 105, 68}  # N57/H70/E72/W99/C102/C105/G68
CORE_READERS = {29, 30}  # H29/K30：碱基阅读器


def count_hbonds(prot_atoms, rna_atoms, rna_donor, rna_acc):
    """返回 (氢键原子对集合, 供体侧计数)。prot_atoms/rna_atoms 为原子 dict 列表。"""
    pairs = set()
    for pa in prot_atoms:
        pname = pa["atom_site.label_atom_id"]
        # 排除 Pro 骨架 N（无 H）
        if pname == "N" and pa["atom_site.label_comp_id"] == "PRO":
```





```
            continue
        pa_coord = U.coord(pa)
        for ra in rna_atoms:
            rname = ra["atom_site.label_atom_id"]
            ra_coord = U.coord(ra)
            dx, dy, dz = (
                pa_coord[0] - ra_coord[0],
                pa_coord[1] - ra_coord[1],
                pa_coord[2] - ra_coord[2],
            )
            d = (dx * dx + dy * dy + dz * dz) ** 0.5
            if d > HB_DIST:
                continue
            if pname in PROT_DONOR and rname in rna_acc:
                pairs.add((pa["atom_site.id"], ra["atom_site.id"]))
            if pname in PROT_ACC and rname in rna_donor:
                pairs.add((pa["atom_site.id"], ra["atom_site.id"]))
    return len(pairs)


def shrake_rupley(coords, radii, probe=1.4, n_sphere=100):
    """计算每个原子的溶剂可及表面积（Å²）。coords: (N,3) float; radii: (N,) float。"""
    import numpy as np

    n = len(coords)
    if n == 0:
        return np.zeros(0)

    # 斐波那契球面点（单位球）
    idx = np.arange(n_sphere)
    z = 1 - 2 * (idx + 0.5) / n_sphere
    phi = np.pi * (1 + 5**0.5) * idx
    xy = np.sqrt(1 - z * z)
    unit = np.stack([xy * np.cos(phi), xy * np.sin(phi), z], axis=1)  # (n_sphere,3)

    r = radii + probe  # (N,)
    # 成对距离矩阵
    diff = coords[:, None, :] - coords[None, :, :]
    dm = np.sqrt((diff**2).sum(-1))  # (N,N)

    asa = np.zeros(n)
    for i in range(n):
        # 邻居 j（与 i 的扩展球可能重叠，且非自身）
        nb = np.where((dm[i] <= r[i] + r) & (np.arange(n) != i))[0]
        if nb.size == 0:
            asa[i] = 4 * np.pi * r[i] ** 2
            continue
        pts = unit * r[i] + coords[i]  # (S,3)
        d = np.sqrt(((pts[:, None, :] - coords[nb][None, :, :]) ** 2).sum(-1))  # (S,M)
        occluded = (d < r[nb][None, :]).any(axis=1)  # (S,)
        asa[i] = 4 * np.pi * r[i] ** 2 * (n_sphere - occluded.sum()) / n_sphere
    return asa


def radii_of(atoms):
    R = {"C": 1.7, "N": 1.55, "O": 1.52, "S": 1.8, "P": 1.8}
    return [R.get(a["atom_site.type_symbol"], 1.7) for a in atoms]


# ============ 读取 manifest 与 qc ============
manifest = []
```





```
with open(os.path.join(OUT_DIR, "manifest.csv"), encoding="utf-8-sig") as f:
    manifest = list(csv.DictReader(f))

skip = set()
with open(os.path.join(OUT_DIR, "qc_report.csv"), encoding="utf-8-sig") as f:
    for row in csv.DictReader(f):
        if row["qc_pass"] == "False":
            skip.add((row["rna"], row["seed"], int(row["rank"])))

# ============ 1) 氢键计数（全部模型） ============
hb_agg = defaultdict(lambda: defaultdict(list))  # rna -> resi -> [per-model hb_base]
hb_full_agg = defaultdict(lambda: defaultdict(list))
resn_map = {}

for row in manifest:
    rna, seed = row["rna"], row["seed"]
    rank = int(row["rank"])
    if (rna, seed, rank) in skip:
        continue
    cols, atoms = U.parse_atom_site(row["path"])
    plus1_all = U.atoms_of(atoms, U.RNA_CHAIN, U.PLUS1_RESI)
    plus1_base = [a for a in plus1_all if a["atom_site.label_atom_id"] in U.BASE_ATOMS]
    plus1_resn = plus1_all[0]["atom_site.label_comp_id"]
    rna_don = RNA_DONOR.get(plus1_resn, set())
    rna_acc = RNA_ACC.get(plus1_resn, set())

    rA = U.residues_by_chain(atoms, U.PROT_CHAIN)
    for resi, patoms in rA.items():
        resn_map[resi] = patoms[0]["atom_site.label_comp_id"]
        hb_agg[rna][resi].append(count_hbonds(patoms, plus1_base, rna_don, rna_acc))
        hb_full_agg[rna][resi].append(count_hbonds(patoms, plus1_all, rna_don, rna_acc))

hb_rows = []
for rna in U.RNA_SEQS:
    for resi in sorted(hb_agg[rna]):
        xs = hb_agg[rna][resi]
        xf = hb_full_agg[rna][resi]
        hb_rows.append(
            {
                "rna": rna,
                "resi": resi,
                "resn": resn_map[resi],
                "n_model": len(xs),
                "mean_hb_base": round(sum(xs) / len(xs), 3),
                "freq_hb_base": round(sum(1 for x in xs if x > 0) / len(xs), 3),
                "mean_hb_full": round(sum(xf) / len(xf), 3),
                "freq_hb_full": round(sum(1 for x in xf if x > 0) / len(xf), 3),
            }
        )

hb_fields = [
    "rna",
    "resi",
    "resn",
    "n_model",
    "mean_hb_base",
    "freq_hb_base",
    "mean_hb_full",
    "freq_hb_full",
]
with open(
```





```
    os.path.join(OUT_DIR, "hbonds_per_residue.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=hb_fields)
    w.writeheader()
    w.writerows(hb_rows)

# ============ 2) 埋藏面积（每 seed rank0 代表结构） ============
import numpy as np  # noqa: E402

area_agg = defaultdict(lambda: defaultdict(list))

for row in manifest:
    if row["rank"] != "0":
        continue
    rna, seed = row["rna"], row["seed"]
    rank = int(row["rank"])
    if (rna, seed, rank) in skip:
        continue
    cols, atoms = U.parse_atom_site(row["path"])

    prot = [a for a in atoms if a["atom_site.label_asym_id"] == U.PROT_CHAIN]
    rna_atoms = [a for a in atoms if a["atom_site.label_asym_id"] == U.RNA_CHAIN]

    all_atoms = prot + rna_atoms
    all_coords = np.array([U.coord(a) for a in all_atoms], dtype=float)
    all_radii = np.array(radii_of(all_atoms), dtype=float)
    prot_coords = np.array([U.coord(a) for a in prot], dtype=float)
    prot_radii = np.array(radii_of(prot), dtype=float)

    asa_complex = shrake_rupley(all_coords, all_radii)
    asa_prot_only = shrake_rupley(prot_coords, prot_radii)

    # 蛋白原子在两种语境下的 ASA，按残基求和求差
    prot_map = defaultdict(lambda: [0.0, 0.0])  # resi -> [asa_complex, asa_free]
    for a, ac, af in zip(prot, asa_complex[: len(prot)], asa_prot_only):
        resi = int(a["atom_site.label_seq_id"])
        prot_map[resi][0] += ac
        prot_map[resi][1] += af
    for resi, (ac, af) in prot_map.items():
        area_agg[rna][resi].append(af - ac)  # 正值 = 被 RNA 埋藏的面积

area_rows = []
for rna in U.RNA_SEQS:
    for resi in sorted(area_agg[rna]):
        xs = area_agg[rna][resi]
        area_rows.append(
            {
                "rna": rna,
                "resi": resi,
                "resn": resn_map[resi],
                "n_model": len(xs),
                "mean_buried_area": round(sum(xs) / len(xs), 2),
            }
        )

area_fields = ["rna", "resi", "resn", "n_model", "mean_buried_area"]
with open(
    os.path.join(OUT_DIR, "buried_area_per_residue.csv"),
    "w",
    newline="",
    encoding="utf-8-sig",
```





```
) as f:
    w = csv.DictWriter(f, fieldnames=area_fields)
    w.writeheader()
    w.writerows(area_rows)

print("氢键表写入 results/hbonds_per_residue.csv")
print("埋藏面积表写入 results/buried_area_per_residue.csv")

# ============ 3) 再次筛选 ============
# 标准：任一组 RNA 中 freq_hb_base>0 或 平均氢键>=0.05，且非禁区
filtered = []
for hb in hb_rows:
    resi = hb["resi"]
    if resi in FORBIDDEN:
        continue
    if (hb["freq_hb_base"] > 0 or hb["mean_hb_base"] >= 0.05) and resi in resn_map:
        filtered.append(hb)

# 合并三组信息
by_resi = defaultdict(dict)
for hb in filtered:
    by_resi[hb["resi"]]["resn"] = hb["resn"]
    by_resi[hb["resi"]][hb["rna"]] = hb["freq_hb_base"]

filter_rows = []
for resi in sorted(by_resi):
    d = by_resi[resi]
    filter_rows.append(
        {
            "resi": resi,
            "resn": d["resn"],
            "freq_hb_G": d.get("GUGCGCG", 0.0),
            "freq_hb_Aswap": d.get("GUGCACG", 0.0),
            "freq_hb_Aoff": d.get("AUGCAGC", 0.0),
            "role": "reader" if resi in CORE_READERS else "",
        }
    )
filter_fields = ["resi", "resn", "freq_hb_G", "freq_hb_Aswap", "freq_hb_Aoff", "role"]
with open(
    os.path.join(OUT_DIR, "filtered_residues.csv"), "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.DictWriter(f, fieldnames=filter_fields)
    w.writeheader()
    w.writerows(filter_rows)

print("\n== 再次筛选后的可设计残基（含 +1 氢键频率 G / Aswap / Aoff）==")
for r in filter_rows:
    print(
        f"  {r['resi']:>3} {r['resn']:>4}  G={r['freq_hb_G']:>5}  "
        f"Aswap={r['freq_hb_Aswap']:>5}  Aoff={r['freq_hb_Aoff']:>5}  {r['role']}"
    )
print("写入 results/filtered_residues.csv")
```





code/historical/code/s4_contacts/10_contact_decomposition.py

```python
# -*- coding: utf-8 -*-
"""
阶段 10 —— 差异接触的「接触类型」分解

对每个与 +1 核苷酸接触的蛋白残基，把接触拆成：
  蛋白侧  ×  RNA 侧
  侧链(sidechain) / 主链(backbone)  ×  碱基(base) / 糖(sugar) / 磷酸(phosphate)

目的：区分「可设计」的碱基特异侧链接触 vs 「结构性」的主链/糖磷酸接触。
  - 侧链–碱基接触 = 可设计（可通过突变侧链调谐碱基特异性）
  - 主链接触      = 结构性（突变侧链改不动骨架，基本不可设计）

产出：results/contact_decomposition.csv
"""
import os
import csv
from collections import defaultdict

import af_utils as U

OUT = "results"

# 蛋白原子分类
BB = {"N", "CA", "C", "O", "OXT"}
# RNA 原子分类
BASE = {"N1", "C2", "N3", "C4", "C5", "C6", "N7", "C8", "N9", "O6", "N2", "N6"}
SUGAR = {"C1'", "C2'", "O2'", "C3'", "O3'", "C4'", "O4'", "C5'", "O5'"}
PHOS = {"P", "OP1", "OP2"}

CUTOFF = 5.0


def classify_prot(name):
    return "bb" if name in BB else "sc"


def classify_rna(name):
    if name in BASE:
        return "base"
    if name in SUGAR:
        return "sugar"
    if name in PHOS:
        return "phos"
    return "other"


def contact_type(prot_atoms, rna_atoms):
    """返回 (has_sc_base, has_bb_base, has_sc_sugphos, has_bb_sugphos)。"""
    sc_base = bb_base = sc_sp = bb_sp = False
    for pa in prot_atoms:
        pcls = classify_prot(pa["atom_site.label_atom_id"])
        pc = U.coord(pa)
        for ra in rna_atoms:
            rcls = classify_rna(ra["atom_site.label_atom_id"])
            rc = U.coord(ra)
            dx, dy, dz = pc[0] - rc[0], pc[1] - rc[1], pc[2] - rc[2]
```





```
            if (dx * dx + dy * dy + dz * dz) ** 0.5 > CUTOFF:
                continue
            if pcls == "sc" and rcls == "base":
                sc_base = True
            elif pcls == "bb" and rcls == "base":
                bb_base = True
            elif pcls == "sc" and rcls in ("sugar", "phos"):
                sc_sp = True
            elif pcls == "bb" and rcls in ("sugar", "phos"):
                bb_sp = True
    return sc_base, bb_base, sc_sp, bb_sp


# 读 manifest（跳过不必要，全部处理）
manifest = list(
    csv.DictReader(open(os.path.join(OUT, "manifest.csv"), encoding="utf-8-sig"))
)

# 聚合：rna -> resi -> 各类型计数
agg = defaultdict(
    lambda: defaultdict(lambda: [0, 0, 0, 0])
)  # [sc_base, bb_base, sc_sp, bb_sp]
nmodel = defaultdict(int)
resn_map = {}

for m in manifest:
    rna = m["rna"]
    nmodel[rna] += 1
    cols, atoms = U.parse_atom_site(m["path"])
    plus1 = U.atoms_of(atoms, U.RNA_CHAIN, U.PLUS1_RESI)
    rA = U.residues_by_chain(atoms, U.PROT_CHAIN)
    for resi, patoms in rA.items():
        resn_map[resi] = patoms[0]["atom_site.label_comp_id"]
        sc_base, bb_base, sc_sp, bb_sp = contact_type(patoms, plus1)
        a = agg[rna][resi]
        a[0] += sc_base
        a[1] += bb_base
        a[2] += sc_sp
        a[3] += bb_sp

# 写 CSV：每 RNA × 残基 的频率
rows = []
for rna in U.RNA_SEQS:
    for resi in sorted(agg[rna]):
        a = agg[rna][resi]
        n = nmodel[rna]
        sc_base = round(a[0] / n, 2)
        bb_base = round(a[1] / n, 2)
        sc_sp = round(a[2] / n, 2)
        bb_sp = round(a[3] / n, 2)
        # 设计性：侧链-碱基接触高 = 可设计特异性；主链为主 = 结构性
        if sc_base >= 0.5 and bb_base < sc_base:
            tag = "可设计(侧链-碱基特异)"
        elif bb_base >= 0.5 or bb_sp >= 0.5:
            tag = "结构性(主链/糖磷酸为主)"
        elif sc_sp >= 0.5:
            tag = "可设计(侧链-糖磷酸)"
        else:
            tag = ""
        rows.append(
            dict(
```





```
                rna=rna,
                resi=resi,
                resn=resn_map[resi],
                sc_base=sc_base,
                bb_base=bb_base,
                sc_sugphos=sc_sp,
                bb_sugphos=bb_sp,
                designability=tag,
            )
        )

with open(f"{OUT}/contact_decomposition.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(
        f,
        fieldnames=[
            "rna",
            "resi",
            "resn",
            "sc_base",
            "bb_base",
            "sc_sugphos",
            "bb_sugphos",
            "designability",
        ],
    )
    w.writeheader()
    w.writerows(rows)

# 聚焦关键/候选残基
KEYS = [
    27,
    28,
    29,
    30,
    31,
    32,
    57,
    58,
    59,
    60,
    67,
    68,
    69,
    70,
    72,
    98,
    99,
    102,
    105,
    129,
    130,
    131,
    132,
    133,
]
print("== 关键残基 接触类型分解（A-swap GUGCACG，频率 0-1）==")
print(
    f"{'resi':>4} {'resn':>4} {'侧链-碱基':>8} {'主链-碱基':>8} {'侧链-糖磷':>8} {'主链-糖磷':>8}  设计性
"
)
for r in rows:
```





```
    if (
        r["rna"] == "GUGCACG"
        and r["resi"] in KEYS
        and (
            r["sc_base"] > 0
            or r["bb_base"] > 0
            or r["sc_sugphos"] > 0
            or r["bb_sugphos"] > 0
        )
    ):
        print(
            f"{r['resi']:>4} {r['resn']:>4} {r['sc_base']:>8.2f} {r['bb_base']:>8.2f} "
            f"{r['sc_sugphos']:>8.2f} {r['bb_sugphos']:>8.2f}  {r['designability']}"
        )

print(f"\n写入 {OUT}/contact_decomposition.csv")
```





code/historical/code/s5_cp/15_cp_dcp_analysis.py

```python
# -*- coding: utf-8 -*-
"""
阶段 15 —— ContactSeek 式 CP + ΔCP 分析（替代距离接触分析）

方法（借鉴 ContactSeek, Nature 2026）：
  - 从 AF3 输出的 full_data JSON 里取「contact_probs」矩阵（两残基在 8 Å 内的概率），
    比「用坐标算距离」更敏感，能捕捉距离没变但接触概率变化的微弱差异。
  - 对每个模型提取「蛋白残基 ↔ +1 核苷酸(chain B resi 5)」的 CP。
  - 按 RNA 分组取平均 CP；算 ΔCP = CP(off) − CP(on)：
        ΔCP_swap = CP(GUGCACG) − CP(GUGCGCG)    （纯 +1 G→A 交换）
        ΔCP_off  = CP(AUGCAGC) − CP(GUGCGCG)    （完整脱靶）
  - ΔCP > 0 = 该残基在 +1=A 时接触增强（脱靶稳定）→ 排A 候选。

产出：results/cp_dcp.csv、results/cp_dcp_ranking.csv
"""
import csv
import json
import os
import re
from collections import defaultdict

import af_utils as U

OUT = "results"

manifest = list(
    csv.DictReader(open(os.path.join(OUT, "manifest.csv"), encoding="utf-8-sig"))
)


def full_data_path(model_cif_path, rank):
    """由 model_0.cif 路径推导 full_data_0.json 路径。"""
    return re.sub(rf"_model_{rank}\.cif$", f"_full_data_{rank}.json", model_cif_path)


def load_cp(model_path, rank):
    """加载 token_chain_ids / token_res_ids / contact_probs。"""
    p = full_data_path(model_path, rank)
    with open(p, encoding="utf-8") as f:
        d = json.load(f)
    return d["token_chain_ids"], d["token_res_ids"], d["contact_probs"]


def extract_cp(chain_ids, res_ids, cp, target_chain, target_res):
    """返回 {蛋白残基: CP(该残基 → 目标核苷酸)}。"""
    # 找目标核苷酸的 token 索引
    tgt_idx = None
    for i in range(len(chain_ids)):
        if chain_ids[i] == target_chain and res_ids[i] == target_res:
            tgt_idx = i
            break
    if tgt_idx is None:
        return {}
    out = {}
    for i in range(len(chain_ids)):
        if chain_ids[i] == U.PROT_CHAIN:
```





```
            # CP 取两方向的最大值（更稳健）
            c = max(cp[i][tgt_idx], cp[tgt_idx][i])
            out[res_ids[i]] = c
    return out


# 聚合：rna -> resi -> [CP 列表]
agg = defaultdict(lambda: defaultdict(list))
nmodel = defaultdict(int)

for m in manifest:
    rna = m["rna"]
    rank = int(m["rank"])
    try:
        chain_ids, res_ids, cp = load_cp(m["path"], rank)
    except FileNotFoundError:
        continue
    nmodel[rna] += 1
    per_res = extract_cp(chain_ids, res_ids, cp, U.RNA_CHAIN, U.PLUS1_RESI)
    for resi, c in per_res.items():
        agg[rna][resi].append(c)

# 平均 CP
mean_cp = defaultdict(dict)  # rna -> resi -> mean CP
for rna in U.RNA_SEQS:
    for resi, cs in agg[rna].items():
        mean_cp[rna][resi] = sum(cs) / len(cs)

# 计算 ΔCP
all_resi = set()
for rna in U.RNA_SEQS:
    all_resi |= set(mean_cp[rna].keys())

rows = []
for resi in sorted(all_resi):
    cg = mean_cp["GUGCGCG"].get(resi, 0.0)
    ca_swap = mean_cp["GUGCACG"].get(resi, 0.0)
    ca_off = mean_cp["AUGCAGC"].get(resi, 0.0)
    rows.append(
        dict(
            resi=resi,
            cp_G=round(cg, 4),
            cp_A_swap=round(ca_swap, 4),
            cp_A_off=round(ca_off, 4),
            dcp_swap=round(ca_swap - cg, 4),
            dcp_off=round(ca_off - cg, 4),
        )
    )

with open(f"{OUT}/cp_dcp.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(
        f, fieldnames=["resi", "cp_G", "cp_A_swap", "cp_A_off", "dcp_swap", "dcp_off"]
    )
    w.writeheader()
    w.writerows(rows)

# 排名：按 ΔCP（swap，纯 +1 判别）降序
ranked = sorted(rows, key=lambda r: -r["dcp_swap"])
with open(f"{OUT}/cp_dcp_ranking.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(
        f,
```





```
        fieldnames=[
            "rank",
            "resi",
            "cp_G",
            "cp_A_swap",
            "cp_A_off",
            "dcp_swap",
            "dcp_off",
        ],
    )
    w.writeheader()
    for i, r in enumerate(ranked, 1):
        w.writerow(dict(rank=i, **r))

# 打印关键残基 + 前 20
KEYS = [27, 28, 29, 30, 31, 32, 57, 58, 59, 60, 70, 72, 98, 99, 102, 105, 130, 132]
print(f"== 关键残基的 CP / ΔCP（+1 核苷酸）==")
print(
    f"{'resi':>4} {'CP(G)':>7} {'CP(A_swap)':>10} {'CP(A_off)':>9} {'ΔCP_swap':>9} {'ΔCP_off':>8}"
)
for resi in KEYS:
    r = next((x for x in rows if x["resi"] == resi), None)
    if r:
        print(
            f"{resi:>4} {r['cp_G']:>7} {r['cp_A_swap']:>10} {r['cp_A_off']:>9} "
            f"{r['dcp_swap']:>+9} {r['dcp_off']:>+8}"
        )

print(f"\n== ΔCP_swap 前 20（+1=G→A 时接触增强最多的残基 = 排A候选）==")
print(f"{'resi':>4} {'CP(G)':>7} {'CP(A)':>7} {'ΔCP':>8}")
for r in ranked[:20]:
    print(f"{r['resi']:>4} {r['cp_G']:>7} {r['cp_A_swap']:>7} {r['dcp_swap']:>+8}")

print(f"\n写入 {OUT}/cp_dcp.csv, {OUT}/cp_dcp_ranking.csv")
```





code/historical/code/s5_cp/19_mut_cp.py

```python
# -*- coding: utf-8 -*-
"""
阶段 19 —— 突变体模型的 CP / ΔCP 分析，并与 WT 对比

对 new/mut/ 下每个突变（× GCG/GCA 两 motif，各 5 模型）：
  提取「蛋白残基 ↔ +1 核苷酸(RNA res 5)」的 CP；
  算 ΔCP = CP(GCA) − CP(GCG)；
  对比 WT 的 ΔCP，看突变是否改变 +1 G/A 判别。

产出：results/mut_cp_dcp.csv（每突变 × 残基的 CP_G/CP_A/ΔCP）
"""
import json
import csv
import os
import glob
from collections import defaultdict

import af_utils as U

OUT = "results"
PLUS1_RESI = 5

# 突变列表（new/mut 下的目录名）
MUTS = [
    "a59p",
    "g25e",
    "g25ek30h",
    "g27d",
    "g27dk30h",
    "g27dk30hq58m",
    "g27t",
    "g27tk30h",
    "g27tk30hq58m",
    "k30h",
    "k30hq58m",
    "k60r",
    "m132d",
    "m132e",
]


def process(dirname):
    cp = defaultdict(list)
    for fjson in sorted(glob.glob(os.path.join(dirname, "*full_data*.json"))):
        d = json.load(open(fjson, encoding="utf-8"))
        cids = d["token_chain_ids"]
        rids = d["token_res_ids"]
        cpm = d["contact_probs"]
        tgt = next(
            i
            for i in range(len(cids))
            if cids[i] == U.RNA_CHAIN and rids[i] == PLUS1_RESI
        )
        for i in range(len(cids)):
            if cids[i] == U.PROT_CHAIN:
                cp[rids[i]].append(max(cpm[i][tgt], cpm[tgt][i]))
```

```
    return {r: sum(v) / len(v) for r, v in cp.items()}


# 读 WT 参考（results/cp_dcp.csv）
wt = {}
for r in csv.DictReader(open(f"{OUT}/cp_dcp.csv", encoding="utf-8-sig")):
    wt[int(r["resi"])] = dict(
        cp_G=float(r["cp_G"]), cp_A=float(r["cp_A_swap"]), dcp=float(r["dcp_swap"])
    )

# 计算每个突变
result = {}
for m in MUTS:
    g = process(f"new/mut/{m}_gcg")
    a = process(f"new/mut/{m}_gca")
    resis = set(g) | set(a)
    result[m] = {
        r: dict(cpG=g.get(r, 0), cpA=a.get(r, 0), dcp=a.get(r, 0) - g.get(r, 0))
        for r in resis
    }

# 输出 CSV
with open(f"{OUT}/mut_cp_dcp.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.writer(f)
    w.writerow(["mut", "resi", "cp_G", "cp_A", "dcp", "wt_cpG", "wt_cpA", "wt_dcp"])
    for m in MUTS:
        for r in sorted(result[m]):
            d = result[m][r]
            ww = wt.get(r, {})
            w.writerow(
                [
                    m,
                    r,
                    round(d["cpG"], 4),
                    round(d["cpA"], 4),
                    round(d["dcp"], 4),
                    round(ww.get("cp_G", 0), 4),
                    round(ww.get("cp_A", 0), 4),
                    round(ww.get("dcp", 0), 4),
                ]
            )

# 打印关键残基的 ΔCP 对比
KEYS = [27, 28, 29, 30, 31, 32, 57, 58, 59, 60, 70, 72, 99, 102, 105, 129, 130, 132]
print("== 各突变的 ΔCP（+1 位，正值=排A方向，负值=保G方向）关键残基 ==")
print(f"{'突变':<16}", end="")
for k in KEYS:
    print(f"{k:>5}", end="")
print(f"{'  (vs WT 变化)':>18}")
for m in MUTS:
    print(f"{m:<16}", end="")
    for k in KEYS:
        d = result[m].get(k, {})
        print(f"{d.get('dcp',0):>+5.2f}", end="")
    # 与 WT 的差异（关键残基 29/30 的 ΔCP 变化）
```





```
    d30 = result[m].get(30, {}).get('dcp', 0) - wt.get(30, {}).get('dcp', 0)
    d29 = result[m].get(29, {}).get('dcp', 0) - wt.get(29, {}).get('dcp', 0)
    print(f"  (ΔΔCP 29={d29:+.2f} 30={d30:+.2f})")
print(f"{'WT(参考)':<16}", end="")
for k in KEYS:
    print(f"{wt.get(k,{}).get('dcp',0):>+5.2f}", end="")
print()
print(f"\n写入 {OUT}/mut_cp_dcp.csv")
```





code/historical/code/s5_cp/52_mutant_baseline_dcp.py

```
# -*- coding: utf-8 -*-
"""
阶段 52 —— 导出「突变体基线 ΔCP（全 199 残基）」完整数据

背景：此前交付的 results/突变体ΔCP原始数据表.xlsx 只含「发生变化的 22 个残基」
      （|ΔΔCP| ≥ 0.05）。本脚本导出全 199 残基的完整 ΔCP / ΔΔCP 矩阵。

数据源：
  results/mut_cp_dcp.csv         batch1，14 个突变体 × 199 残基
                                 列：mut, resi, cp_G, cp_A, dcp, wt_cpG, wt_cpA, wt_dcp
  results/mut_cp_dcp_batch2.csv  batch2，7 个突变体 + 独立 WT 重跑 × 199 残基
                                 列：mut, resi, cp_G, cp_A, dcp, wt_dcp

定义：
  ΔCP = CP(GCA) − CP(GCG)，CP = 「该残基 ↔ +1 位核苷酸」的 AF3 接触概率
  ΔΔCP = ΔCP(突变体) − ΔCP(WT)
  正 ΔΔCP = 换成 A 后接触增强（偏向脱靶）；负 = 减弱

产出：results/突变体基线ΔCP_全199残基.xlsx
      1_ΔCP矩阵          199 残基 × (WT基线 + 21 突变体)
      2_ΔΔCP矩阵         199 残基 × 21 突变体
      3_长格式明细        每行 = 一个(突变体, 残基)
      4_WT重复性参考      batch2 独立 WT 重跑 vs 基线 → AF3 自身噪声量级
      5_汇总
"""
import csv, sys, collections

sys.stdout.reconfigure(encoding="utf-8")
import numpy as np
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side
from openpyxl.utils import get_column_letter

NAVY, LIGHT, ACC = "1F3B66", "F2F4F8", "B44A3C"
RED = PatternFill("solid", fgColor="F8D7DA")  # ΔΔCP ≥ +0.15
BLU = PatternFill("solid", fgColor="D6E4F7")  # ΔΔCP ≤ −0.10
HDR = PatternFill("solid", fgColor=NAVY)
THIN = Border(*[Side(style="thin", color="BFBFBF")] * 4)
N = 199

WT_SEQ = (
    "MEASPASGPRHLMDPAIFTSNFNNGIGRHKTYLCYEVERLDNGTSVKMDQHRGFLHNQAKNLLCGFYGRHVEVNFIKKFTSERDF"
    "HPSMSCSITWFLSWSPCWECSQAIREFLSRHPGVTLVIYVARLFWHMDQQNRQGLRDLVNSGVTIQIMTYDEFKHCWDTFVDHQG"
    "APFQPWDGLDEHSQALSGRLRAILQNQGN"
)
assert len(WT_SEQ) == N, f"序列长度 {len(WT_SEQ)} != {N}"
AA = {i + 1: a for i, a in enumerate(WT_SEQ)}

# ---------- 读数据 ----------
b1 = list(csv.DictReader(open("results/mut_cp_dcp.csv", encoding="utf-8-sig")))
b2 = list(csv.DictReader(open("results/mut_cp_dcp_batch2.csv", encoding="utf-8-sig")))

# 突变体展示名
NAME = {
    "a59p": "A59P",
    "g25e": "G25E",
```





```
    "g25ek30h": "G25E-K30H",
    "g27d": "G27D",
    "g27dk30h": "G27D-K30H",
    "g27dk30hq58m": "G27D-K30H-Q58M",
    "g27t": "G27T",
    "g27tk30h": "G27T-K30H",
    "g27tk30hq58m": "G27T-K30H-Q58M",
    "k30h": "K30H",
    "k30hq58m": "K30H-Q58M",
    "k60r": "K60R",
    "m132d": "M132D",
    "m132e": "M132E",
    "q58m": "Q58M",
    "r28a": "R28A",
    "r28s": "R28S",
    "t31i": "T31I",
    "t31l": "T31L",
    "t31v": "T31V",
    "y32f": "Y32F",
}
ORDER = [
    "g25e",
    "g25ek30h",
    "g27t",
    "g27d",
    "g27tk30h",
    "g27dk30h",
    "g27tk30hq58m",
    "g27dk30hq58m",
    "k30h",
    "k30hq58m",
    "q58m",
    "r28a",
    "r28s",
    "t31v",
    "t31i",
    "t31l",
    "y32f",
    "a59p",
    "k60r",
    "m132d",
    "m132e",
]

# 基线（三文件已校验逐残基一致）
WT = np.array([float(r["wt_dcp"]) for r in b1 if r["mut"] == "g25e"])

DCP, WTCP = {}, {}
for r in b1:
    DCP.setdefault(r["mut"], {})[int(r["resi"])] = float(r["dcp"])
    WTCP.setdefault(r["mut"], {})[int(r["resi"])] = (
        float(r["wt_cpG"]),
        float(r["wt_cpA"]),
    )
WTRUN2 = None
for r in b2:
    if r["mut"] == "wt":
        WTRUN2 = DCP.get("__wt2__", {})
    DCP.setdefault(r["mut"], {})[int(r["resi"])] = float(r["dcp"])

MUTS = [m for m in ORDER if m in DCP]
```





```
print(f"突变体 {len(MUTS)} 个，基线残基数 {len(WT)}")

X = np.array([[DCP[m][i] for i in range(1, N + 1)] for m in MUTS])  # (M,199) ΔCP
D = X - WT[None, :]  # ΔΔCP


def fm(v, nd=4):
    return round(float(v), nd)


# ---------- 写 Excel ----------
wb = Workbook()


def style_header(ws, ncol, widths):
    for c in range(1, ncol + 1):
        cell = ws.cell(1, c)
        cell.font = Font(bold=True, color="FFFFFF", size=10)
        cell.fill = HDR
        cell.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
    for i, w in enumerate(widths, 1):
        ws.column_dimensions[get_column_letter(i)].width = w
    ws.freeze_panes = "D2"


def paint(ws, r0, c0, nr, nc, arr, hi=0.15, lo=-0.10):
    for i in range(nr):
        for j in range(nc):
            v = arr[i, j]
            if v >= hi:
                ws.cell(r0 + i, c0 + j).fill = RED
            elif v <= lo:
                ws.cell(r0 + i, c0 + j).fill = BLU


# --- Sheet1 ΔCP 矩阵 ---
ws = wb.active
ws.title = "1_ΔCP矩阵"
ws.append(["残基", "WT", "区域"] + [NAME.get(m, m) for m in MUTS])
for i in range(1, N + 1):
    ws.append([i, AA[i], ""] + [fm(DCP[m][i]) for m in MUTS])
style_header(ws, 3 + len(MUTS), [7, 6, 10] + [11] * len(MUTS))
for r in range(2, N + 2):
    ws.cell(r, 1).font = Font(bold=True, size=9)
    ws.cell(r, 2).font = Font(bold=True, size=9)
ws.cell(
    N + 3, 1, "ΔCP = CP(GCA) − CP(GCG)；CP 为 AF3「该残基 ↔ +1 位核苷酸」接触概率"
).font = Font(italic=True, size=9)

# --- Sheet2 ΔΔCP 矩阵 ---
ws2 = wb.create_sheet("2_ΔΔCP矩阵")
ws2.append(["残基", "WT", "区域"] + [NAME.get(m, m) for m in MUTS])
for i in range(1, N + 1):
    ws2.append([i, AA[i], ""] + [fm(D[k, i - 1]) for k, m in enumerate(MUTS)])
style_header(ws2, 3 + len(MUTS), [7, 6, 10] + [11] * len(MUTS))
paint(ws2, 2, 4, N, len(MUTS), D.T)
for r in range(2, N + 2):
    ws2.cell(r, 1).font = Font(bold=True, size=9)
    ws2.cell(r, 2).font = Font(bold=True, size=9)
ws2.cell(
    N + 3,
```





```
    1,
    "ΔΔCP = ΔCP(突变体) − ΔCP(WT)；红 ≥ +0.15（偏脱靶），蓝 ≤ −0.10（偏 on-target）",
).font = Font(italic=True, size=9)

# --- Sheet3 长格式 ---
ws3 = wb.create_sheet("3_长格式明细")
ws3.append(
    [
        "突变体",
        "残基",
        "WT_aa",
        "CP_GCG",
        "CP_GCA",
        "ΔCP_突变体",
        "ΔCP_WT",
        "ΔΔCP",
        "|ΔΔCP|≥0.05",
    ]
)
nch = 0
for k, m in enumerate(MUTS):
    for i in range(1, N + 1):
        g, a = WTCP.get(m, {}).get(i, (None, None))
        if g is None:  # batch2 无 wt_cpG/wt_cpA 列，用 dcp 反推：
            g, a = "", ""  # CP_GCG/CP_GCA 留空（batch2 未导出分项）
        dd = D[k, i - 1]
        flag = "是" if abs(dd) >= 0.05 else ""
        if flag:
            nch += 1
        ws3.append(
            [NAME.get(m, m), i, AA[i], g, a, fm(DCP[m][i]), fm(WT[i - 1]), fm(dd), flag]
        )
style_header(ws3, 9, [16, 7, 7, 10, 10, 13, 11, 11, 15])
ws3.auto_filter.ref = f"A1:I{ws3.max_row}"
print(f"长格式行数 {ws3.max_row - 1}，其中 |ΔΔCP|≥0.05 的 {nch} 行")

# --- Sheet4 WT 重复性参考 ---
ws4 = wb.create_sheet("4_WT重复性参考")
b2wt = [float(r["dcp"]) for r in b2 if r["mut"] == "wt"]
ws4.append(
    ["残基", "WT_aa", "ΔCP_基线(AF3批1)", "ΔCP_WT独立重跑(AF3批2)", "差值(=AF3自身噪声)"]
)
for i in range(1, N + 1):
    ws4.append([i, AA[i], fm(WT[i - 1]), fm(b2wt[i - 1]), fm(b2wt[i - 1] - WT[i - 1])])
style_header(ws4, 5, [7, 7, 20, 24, 20])
dif = np.array(b2wt) - WT
ws4.cell(
    N + 3,
    1,
    f"均值 {dif.mean():+.4f}   SD {dif.std(ddof=1):.4f}   最大|差| {np.abs(dif).max():.4f}",
).font = Font(bold=True, size=9)
ws4.cell(
    N + 4,
    1,
    "同一 WT 序列、不同 AF3 作业之间的差异 = 本方法噪声下限；小于此量级的 ΔΔCP 不可解读",
).font = Font(italic=True, size=9)

# --- Sheet5 汇总 ---
ws5 = wb.create_sheet("5_汇总")
ws5.append(
    [
```





```
        "突变体",
        "残基数",
        "|ΔΔCP|≥0.05 的残基数",
        "|ΔΔCP|≥0.10",
        "max|ΔΔCP|",
        "mean ΔCP(突变体)",
        "mean ΔCP(WT)",
        "ΣCP_GCG",
        "ΣCP_GCA",
    ]
)
for k, m in enumerate(MUTS):
    dd = D[k]
    dcp_m = np.array([DCP[m][i] for i in range(1, N + 1)])
    ws5.append(
        [
            NAME.get(m, m),
            N,
            int((np.abs(dd) >= 0.05).sum()),
            int((np.abs(dd) >= 0.10).sum()),
            fm(np.abs(dd).max()),
            fm(dcp_m.mean()),
            fm(WT.mean()),
            fm(dcp_m[dcp_m > 0].sum()),
            fm((-dcp_m[dcp_m < 0]).sum()),
        ]
    )
ws5.append(["（WT 基线）", N, 0, 0, 0, fm(WT.mean(), 4), fm(WT.mean(), 4), "", ""])
style_header(ws5, 9, [16, 8, 20, 15, 12, 17, 15, 11, 11])
ws5.auto_filter.ref = f"A1:I{ws5.max_row}"

wb.save("results/突变体基线ΔCP_全199残基.xlsx")
print("写入 results/突变体基线ΔCP_全199残基.xlsx")

# ---------- 控制台核对 ----------
print("\n" + "=" * 100)
print("各突变体相对 WT 基线的 ΔΔCP 分布（全 199 残基）")
print("=" * 100)
print(
    f"{'突变体':<18}{'|ΔΔ|≥0.05':>10}{'|ΔΔ|≥0.10':>10}{'max|ΔΔ|':>10}"
    f"{'max(ΔΔ)':>10}{'min(ΔΔ)':>10}{'ΔΔ总量':>10}"
)
for k, m in enumerate(MUTS):
    dd = D[k]
    print(
        f"{NAME.get(m, m):<18}{int((np.abs(dd)>=0.05).sum()):>10}{int((np.abs(dd)>=0.10).sum()):>10}"
        f"{np.abs(dd).max():>10.3f}{dd.max():>+10.3f}{dd.min():>+10.3f}{dd.sum():>+10.3f}"
    )

print("\n【AF3 自身噪声参考】WT 独立重跑 vs 基线：")
print(
    f"  mean={dif.mean():+.4f}  SD={dif.std(ddof=1):.4f}  max|差|={np.abs(dif).max():.4f}"
)
print(f"  → |ΔΔCP| < {np.abs(dif).max():.3f} 的残基差异不可解读")
```





code/historical/code/s7_wetlab/09_experimental_validation.py

```python
# -*- coding: utf-8 -*-
"""
阶段 9 —— 学姐第一批单位点饱和实验数据 重新分析 + 计算预测交叉验证

数据：ACA_295(脱靶) / GCG_388(on-target) / GCA_871(+1=A脱靶)，每个突变 2 重复。
指标：
  on-target 保留率 = GCG_mut / GCG_WT
  +1 比(保G排A) = GCG_388 / GCA_871    (WT=1.37，越高越特异)
产出：results/experimental_ranking.csv、results/cross_validation.csv
"""
import csv
import statistics
import openpyxl

SRC = r"data\external\01a06031-d7d4-72cf-a2a5-f46d90d768dd.xlsx"
OUT = "results"

wb = openpyxl.load_workbook(SRC, data_only=True)
ws = wb["Sheet1"]
dp = wb["测序深度"]

data, depth = {}, {}
for row in ws.iter_rows(values_only=True):
    if row[0] is None or row[1] is None:
        continue
    n = str(row[0]).strip()
    if n == "ACA_295":
        continue
    data.setdefault(n, []).append((row[1], row[2], row[3]))
for row in dp.iter_rows(values_only=True):
    if row[0] is None or row[1] is None:
        continue
    n = str(row[0]).strip()
    if n == "ACA_295_DP":
        continue
    depth.setdefault(n, []).append((row[1], row[2], row[3]))


def mean(xs):
    return statistics.mean(xs)


wt = next(mean([r[1] for r in reps]) for n, reps in data.items() if "WT" in n)
wt_gca = next(mean([r[2] for r in reps]) for n, reps in data.items() if "WT" in n)

rows = []
for n, reps in data.items():
    gcg = mean([r[1] for r in reps])
    gca = mean([r[2] for r in reps])
    aca = mean([r[0] for r in reps])
    ratio = gcg / gca if gca else 0
    retain = gcg / wt
    d = depth.get(n)
    mindp = min(min(x) for x in d) if d else None
    rows.append(
        dict(
```





```
            mut=n,
            GCG=round(gcg, 3),
            GCA=round(gca, 3),
            ACA=round(aca, 3),
            ratio=round(ratio, 2),
            retain=round(retain, 2),
            depth=mindp,
            active=retain >= 0.7,
        )
    )

rows.sort(key=lambda r: (-r["ratio"]))

# 全排名（保留活跃） + 探针（活性崩但高比）
with open(f"{OUT}/experimental_ranking.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(
        f, fieldnames=["mut", "GCG", "GCA", "ACA", "ratio", "retain", "depth", "active"]
    )
    w.writeheader()
    for r in rows:
        w.writerow(r)

# 交叉验证
CAND = [
    ("M132E", "132E"),
    ("M132D", "132D"),
    ("K30H", "30H"),
    ("K30N", "30N"),
    ("G27D", "27D"),
    ("G27T", "27T"),
    ("G27V", "27V"),
    ("G27I", "27I"),
    ("T31A", "31A"),
    ("T31C", "31C"),
    ("A59S", "59S"),
    ("K60H", "60H"),
    ("R28W", "28W"),
    ("25E", "25E"),
    ("58M", "58M"),
]
cv = []
for label, key in CAND:
    if key not in data:
        cv.append(dict(cand=label, exp="未测试", ratio="", retain="", verdict="未覆盖"))
        continue
    reps = data[key]
    gcg = mean([r[1] for r in reps])
    gca = mean([r[2] for r in reps])
    ratio = gcg / gca if gca else 0
    retain = gcg / wt
    verdict = (
        "✓证实"
        if (ratio > wt / wt_gca and retain >= 0.7)
        else ("✗证伪/无改善" if ratio <= wt / wt_gca else "活性偏低(探针)")
    )
    cv.append(
        dict(
            cand=label,
            exp=key,
            ratio=round(ratio, 2),
            retain=f"{retain:.0%}",
```





```
            verdict=verdict,
        )
    )

with open(f"{OUT}/cross_validation.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=["cand", "exp", "ratio", "retain", "verdict"])
    w.writeheader()
    w.writerows(cv)

print(f"WT 基线: GCG={wt:.3f} GCA={wt_gca:.3f}  +1比={wt/wt_gca:.2f}")
print(f"\n=== 活跃(保留≥70%)突变按 +1比 排序 前 15 ===")
for r in [x for x in rows if x["active"]][:15]:
    print(
        f"  {r['mut']:<8} GCG={r['GCG']:.3f} GCA={r['GCA']:.3f} ACA={r['ACA']:.3f} "
        f"+1比={r['ratio']:.2f} 保留={r['retain']:.0%}"
    )
print(f"\n=== 计算候选 交叉验证 ===")
for r in cv:
    print(
        f"  {r['cand']:<8} 实验{r['exp']:<5} +1比={r['ratio']:<5} 保留={r['retain']:<5} {r['verdict'
]}"
    )
print(f"\n写入 {OUT}/experimental_ranking.csv, {OUT}/cross_validation.csv")
```





code/historical/code/s7_wetlab/30_GCG_vs_off_WT.py

```python
# -*- coding: utf-8 -*-
"""
阶段 30 —— 靶点(GCG)编辑总和 vs 脱靶编辑总和 排序与绘图（P 部分仅保留 WT）

样本范围变更（相对阶段 29）：
  剔除：P4-SNE、P5-TWD、P6-WFD、P7-THE、P7-SNE、P8-GVD（P 系列 loop 重设计变体）
  保留：PUF12-9 —— 即 WT / 组合亲本，作为正常一行参与排序（图中以星标与绿色突出）

口径（与阶段 29 一致）：
  靶点 = 全部 47 个 GCG 位点  → Σ_gcg
  脱靶 = 其余 244 个 C 位点    → Σ_off
  靶向比 = Σ_gcg / Σ_off      → 排序依据
  活性   = C388 / C388(WT=PUF12-9)

产出：results/GCG_vs_off_ranking_WT.csv
      results/GCG_vs_off_ranking_WT.png
"""
import glob, os, re, sys, csv, statistics
from collections import defaultdict

sys.stdout.reconfigure(encoding="utf-8")

REF = "".join(
    l
    for l in open("Seq/Bam/G22609034389-1-32F.for.ref.fa", encoding="utf-8")
    .read()
    .splitlines()
    if not l.startswith(">")
)
L = len(REF)
FLOOR = 0.05
CS = [i + 1 for i, c in enumerate(REF) if c == "C"]


def motif(p):
    i = p - 1
    return "?" if (i - 1 < 0 or i + 1 >= L) else REF[i - 1] + "C" + REF[i + 1]


GCG = [p for p in CS if motif(p) == "GCG"]
NON = [p for p in CS if motif(p) != "GCG"]
WT_NAME = "PUF12-9"
EXCLUDE_PS = {"P4-SNE", "P5-TWD", "P6-WFD", "P7-THE", "P7-SNE", "P8-GVD"}


def parse(p):
    out = {}
    for i, l in enumerate(open(p, encoding="latin-1")):
        if i == 0:
            continue
        c = l.rstrip("\n").split("\t")
        if len(c) < 11 or c[2] != "C" or c[3] != "T":
            continue
        try:
            pos, freq = int(c[1]), float(c[7])
        except ValueError:
```





```
            continue
        if freq >= FLOOR:
            out[pos] = freq
    return out


def mut_of(n):
    m = re.match(r'G\d+-(b?)(\d+)-(.*)$', n)
    return m.group(3) if m else n


by_mut = defaultdict(list)
for f in sorted(glob.glob("Seq/Var/*.var.xls")):
    if ".filt." in f:
        continue
    n = os.path.basename(f).replace(".var.xls", "")
    by_mut[mut_of(n)].append(parse(f))

PAR_C388 = statistics.mean([st.get(388, 0) for st in by_mut[WT_NAME]])
print(f"WT(PUF12-9) 的 C388 = {PAR_C388:.4f}  → 活性 = C388 / {PAR_C388:.5f}")
print(f"剔除的 P 系列: {', '.join(sorted(EXCLUDE_PS))}；保留 WT: {WT_NAME}\n")

rows = []
for mut, ss in by_mut.items():
    if mut in EXCLUDE_PS:
        continue
    sg = statistics.mean([sum(st.get(p, 0.0) for p in GCG) for st in ss])
    so = statistics.mean([sum(st.get(p, 0.0) for p in NON) for st in ss])
    n_g = statistics.mean([sum(1 for p in GCG if p in st) for st in ss])
    n_o = statistics.mean([sum(1 for p in NON if p in st) for st in ss])
    c388 = statistics.mean([st.get(388, 0) for st in ss])
    rows.append(
        dict(
            mut=mut,
            sum_gcg=sg,
            sum_off=so,
            ratio=(sg / so) if so > 0 else float("inf"),
            n_gcg=n_g,
            n_off=n_o,
            c388=c388,
            act=c388 / PAR_C388,
            is_wt=(mut == WT_NAME),
        )
    )

main = sorted(rows, key=lambda r: -r["ratio"])

print("=" * 122)
print("表  靶点(GCG)编辑总和 / 脱靶编辑总和 排序（P 部分仅保留 WT；按靶向比降序）")
print("=" * 122)
print(
    f"{'排名':>4}{'突变':<20}{'Σ_GCG(靶点)':>12}{'Σ_off(脱靶)':>12}{'靶向比':>9}"
    f"{'活性(保留)':>11}{'C388':>8}{'GCG位点数':>10}"
)
rk = 0
for r in main:
    if r["sum_gcg"] <= 0.01:
        continue
    rk += 1
    tag = "  ← WT 基线" if r["is_wt"] else ""
    print(
```

```
        f"{rk:>4}{r['mut']:<20}{r['sum_gcg']*100:>12.0f}{r['sum_off']*100:>12.0f}"
        f"{r['ratio']:>9.2f}{r['act']*100:>10.0f}%{r['c388']*100:>8.0f}{r['n_gcg']:>10.1f}{tag}"
    )

print()
print("=== 联合判据：活性 ≥70% 的候选（按靶向比降序）===")
print(f"{'突变':<20}{'靶向比':>9}{'活性':>8}{'Σ_GCG':>9}{'Σ_off':>8}")
for r in sorted(
    [x for x in main if x["act"] >= 0.70 and x["sum_gcg"] > 0.01],
    key=lambda x: -x["ratio"],
):
    print(
        f"{r['mut']:<20}{r['ratio']:>9.2f}{r['act']*100:>7.0f}%{r['sum_gcg']*100:>9.0f}{r['sum_off
']*100:>8.0f}"
    )

with open(
    "results/GCG_vs_off_ranking_WT.csv", "w", newline="", encoding="utf-8-sig"
) as f:
    w = csv.writer(f)
    w.writerow(
        [
            "rank",
            "mutation",
            "is_WT",
            "sum_GCG_pct",
            "sum_nonGCG_pct",
            "targeting_ratio",
            "activity_retention",
            "C388_pct",
            "n_GCG_detected",
            "n_nonGCG_detected",
        ]
    )
    rk = 0
    for r in main:
        if r["sum_gcg"] > 0.01:
            rk += 1
        w.writerow(
            [
                rk if r["sum_gcg"] > 0.01 else "",
                r["mut"],
                r["is_wt"],
                f"{r['sum_gcg']*100:.1f}",
                f"{r['sum_off']*100:.1f}",
                f"{r['ratio']:.3f}",
                f"{r['act']:.3f}",
                f"{r['c388']*100:.2f}",
                f"{r['n_gcg']:.1f}",
                f"{r['n_off']:.1f}",
            ]
        )
print("\n写入 results/GCG_vs_off_ranking_WT.csv")

# ---------------- 绘图 ----------------
try:
    import matplotlib

    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    import numpy as np
```





```
    plt.rcParams["font.sans-serif"] = ["Microsoft YaHei", "SimHei", "DejaVu Sans"]
    plt.rcParams["axes.unicode_minus"] = False

    plot_rows = [r for r in main if r["sum_gcg"] > 0.01][::-1]
    names = [r["mut"] + ("  ★WT" if r["is_wt"] else "") for r in plot_rows]
    sg = np.array([r["sum_gcg"] * 100 for r in plot_rows])
    so = np.array([r["sum_off"] * 100 for r in plot_rows])
    act = np.array([r["act"] for r in plot_rows])
    is_wt = np.array([r["is_wt"] for r in plot_rows])

    fig = plt.figure(figsize=(19, 13.5), dpi=150)
    gs = fig.add_gridspec(1, 2, width_ratios=[1.25, 1], wspace=0.30)

    # 面板A：成对柱，WT 用绿色高亮
    ax = fig.add_subplot(gs[0, 0])
    y = np.arange(len(names))
    h = 0.38
    cols_g = ["#1E7A46" if w else "#2E548B" for w in is_wt]
    cols_o = ["#7DCEA0" if w else "#C0392B" for w in is_wt]
    ax.barh(y + h / 2, sg, height=h, color=cols_g, label="靶点 GCG 编辑总和 Σ_gcg")
    ax.barh(y - h / 2, so, height=h, color=cols_o, label="脱靶(其他 C)编辑总和 Σ_off")
    for i, r in enumerate(plot_rows):
        lab = f"比 {r['ratio']:.1f}  活性 {r['act']*100:.0f}%" + (
            "  ★WT基线" if r["is_wt"] else ""
        )
        ax.text(
            max(sg[i], so[i]) + 12,
            i,
            lab,
            va="center",
            fontsize=7.5,
            color="#1E7A46" if r["is_wt"] else "#333333",
            fontweight="bold" if r["is_wt"] else "normal",
        )
    ax.set_yticks(y)
    ax.set_yticklabels(names, fontsize=8)
    for tick, w in zip(ax.get_yticklabels(), is_wt):
        if w:
            tick.set_color("#1E7A46")
            tick.set_fontweight("bold")
    ax.set_xlabel("编辑频率总和 %（47 个 GCG 靶点 / 244 个非 GCG 位点）", fontsize=11)
    ax.set_title(
        "各突变体的靶点编辑总和 vs 脱靶编辑总和\n"
        "（自下而上 = 靶向比 Σ_gcg/Σ_off 由低到高；P 部分仅保留 WT）",
        fontsize=12,
    )
    ax.legend(loc="lower right", fontsize=9)
    ax.grid(alpha=0.25, axis="x")

    # 面板B：散点
    ax2 = fig.add_subplot(gs[0, 1])
    nw = ~is_wt
    sc = ax2.scatter(
        so[nw],
        sg[nw],
        c=act[nw] * 100,
        cmap="RdYlGn",
        s=70,
        vmin=0,
        vmax=90,
```





```
        edgecolors="#333333",
        linewidths=0.6,
        zorder=3,
    )
    ax2.scatter(
        so[is_wt],
        sg[is_wt],
        marker="*",
        s=420,
        c="#1E7A46",
        edgecolors="white",
        linewidths=1.2,
        zorder=5,
        label="WT (PUF12-9)",
    )
    lim = max(so.max(), sg.max()) * 1.12
    ax2.plot([0, lim], [0, lim], "--", c="#7F8C8D", lw=1.2)
    ax2.text(
        lim * 0.63, lim * 0.70, "Σ_gcg = Σ_off", fontsize=8, color="#7F8C8D", rotation=32
    )
    for r in plot_rows:
        if r["ratio"] > 3 or r["act"] > 0.7 or r["sum_off"] > 300 or r["is_wt"]:
            ax2.annotate(
                r["mut"] + ("(WT)" if r["is_wt"] else ""),
                (r["sum_off"] * 100, r["sum_gcg"] * 100),
                textcoords="offset points",
                xytext=(6, 4),
                fontsize=7.5,
                color="#1E7A46" if r["is_wt"] else "#333333",
                fontweight="bold" if r["is_wt"] else "normal",
            )
    cb = fig.colorbar(sc, ax=ax2, fraction=0.04, pad=0.02)
    cb.set_label("活性（相对 WT 的 C388 保留 %）", fontsize=10)
    ax2.set_xlabel("脱靶编辑总和 Σ_off %", fontsize=11)
    ax2.set_ylabel("靶点 GCG 编辑总和 Σ_gcg %", fontsize=11)
    ax2.set_title(
        "靶点 vs 脱靶 编辑总和\n（左上角 = 高靶向编辑 + 低脱靶；颜色 = 活性）",
        fontsize=12,
    )
    ax2.legend(loc="lower right", fontsize=9)
    ax2.grid(alpha=0.25)

    fig.suptitle(
        "以「全部 GCG 为靶点」：靶点/脱靶编辑总和排序（P 部分仅保留 WT）",
        fontsize=15,
        y=0.975,
    )
    fig.savefig("results/GCG_vs_off_ranking_WT.png", bbox_inches="tight")
    print("写入 results/GCG_vs_off_ranking_WT.png")
except Exception as e:
    print("绘图跳过:", e)
```





code/historical/code/s7_wetlab/35_dry_vs_wet_ranking.py

```python
# -*- coding: utf-8 -*-
"""
阶段 35 —— 干实验预测 vs 湿实验验证 最佳候选排序对照表
用途：把「干预测排出来的名次」与「湿实验实测的名次」并置，量化各预测手段的可靠性。

三张表：
  Sheet1 单点/突变级对照（干预测排名 vs 湿实测排名 + 一致性）
  Sheet2 位点级对照（16 个已测位点的干预测依据 vs 湿最佳表现）
  Sheet3 预测方法可靠性汇总（每种手段与实测的一致率）

产出：results/干湿排序对照表.xlsx / .csv
"""
import csv, re, sys, statistics

sys.stdout.reconfigure(encoding="utf-8")
from openpyxl import Workbook
from openpyxl.styles import Font, PatternFill, Alignment, Border, Side

NAVY = "1F3B66"
LIGHT = "F2F4F8"
OK = "E6F4EA"
BAD = "FBE3E1"
MID = "FFF6E0"


# ---------- 载入 ----------
def load(p):
    return list(csv.DictReader(open(p, encoding="utf-8-sig")))


mcsm = {r["mut"]: float(r["combined"]) for r in load("results/mcsm_single_rerank.csv")}
prem = {r["mut"]: float(r["score"]) for r in load("results/prempri_single_rerank.csv")}
wet = {}
for r in load("results/experimental_ranking.csv"):
    m = re.match(r'(\d+)([A-Z])$', r["mut"])
    if m:
        wet[r["mut"]] = dict(
            resi=int(m.group(1)),
            ratio=float(r["ratio"]),
            retain=float(r["retain"]),
            active=r["active"] == "True",
        )
# 组合实测
combo = {}
for r in load("results/offtarget_screen_final.csv"):
    try:
        combo[r["mutant"]] = dict(
            on=float(r["on_388"]),
            load=float(r["off_load_sum"]),
            norm=float(r["norm_load"]),
            S=float(r["S_global"]),
        )
    except ValueError:
        pass
# 组合干预测（相乘模型）
pred_combo = {}
```





```
for r in load("results/combinations_top1000.csv"):
    pred_combo[r["mutations"].replace("+", "-").replace("G", "G")] = float(r["est_ratio"])
# GCG 口径
gcg = {}
for r in load("results/GCG_vs_off_ranking_WT.csv"):
    try:
        gcg[r["mutation"]] = dict(
            ratio=float(r["targeting_ratio"]),
            act=float(r["activity_retention"]),
            is_wt=r["is_WT"] == "True",
        )
    except ValueError:
        pass


# 1-letter 转换（干预测键名 T31V 等；湿实验键名 31A 等）
def to_wet_key(k):
    m = re.match(r'([A-Z])(\d+)([A-Z])$', k)
    return f"{m.group(2)}{m.group(3)}" if m else k


# ---------- Sheet1：突变级对照 ----------
rows1 = []
for mut_dry, v_mcsm in mcsm.items():
    wk = to_wet_key(mut_dry)
    if wk not in wet:
        continue
    rows1.append(
        dict(
            dry=mut_dry,
            wet_key=wk,
            resi=wet[wk]["resi"],
            mcsm=v_mcsm,
            prem=prem.get(mut_dry),
            ratio=wet[wk]["ratio"],
            retain=wet[wk]["retain"],
            active=wet[wk]["active"],
        )
    )
# 排名（干：mCSM 降序；湿：+1比 降序）
rows1.sort(key=lambda x: -x["mcsm"])
for i, r in enumerate(rows1, 1):
    r["mcsm_rank"] = i
tmp = sorted(rows1, key=lambda x: -x["ratio"])
for i, r in enumerate(tmp, 1):
    r["wet_rank"] = i
for r in rows1:
    r["rank_diff"] = r["wet_rank"] - r["mcsm_rank"]
    d = r["rank_diff"]
    if d >= 3:
        r["verdict"] = f"✗ 干高估（干第{r['mcsm_rank']}→湿第{r['wet_rank']}）"
    elif d <= -3:
        r["verdict"] = f"↑ 干低估（干第{r['mcsm_rank']}→湿第{r['wet_rank']}）"
    else:
        r["verdict"] = f"○ 大致一致（差 {abs(d)} 名）"

# ---------- Sheet2：位点级对照 ----------
# 干预测依据（来自各分析）
SITE = {
    25: ("loop1-N", "可动", "0.02", "MI 1.07（与K30，最高）", "mCSM 中性", "-"),
    26: ("loop1-N", "可动", "0.04", "MI 0.90（与K30）", "mCSM 弱", "-"),
```





```
    27: (
        "loop1–N（–1锚）",
        "慎动",
        "0.82",
        "–",
        "mCSM 无法评分(Gly)",
        "PremPRI 弱(+0.13)",
    ),
    28: (
        "loop1–N（壁）",
        "慎动",
        "0.84",
        "–",
        "mCSM 判别强(swap +1.52)",
        "PremPRI 优保G(+0.23)",
    ),
    29: ("+1 阅读器", "可动", "0.22", "–", "★ mCSM swap 最强(+5.36)", "–"),
    30: (
        "+1 阅读器（主）",
        "可动",
        "0.36",
        "MI 0.87（与T31）",
        "★ mCSM swap +2.89",
        "PremPRI 中性(+0.03)",
    ),
    31: (
        "+1 阅读器（A侧）",
        "慎动",
        "0.82",
        "MI 0.87（与K30）",
        "mCSM 不可靠(RSA=0)",
        "PremPRI 支持排A(+0.29)",
    ),
    32: ("±2 位", "可动", "0.56", "–", "mCSM +1口径不利", "PremPRI 判别但伤G(+0.23)"),
    57: ("loop3（催化必需）", "禁区", "0.67", "共进化 MI 最高（0.30）", "–", "–"),
    58: ("loop3（排A）", "可动", "0.29", "MI 0.77（与30H）", "–", "–"),
    59: (
        "loop3（壁）",
        "可动",
        "0.16",
        "MI 0.88（与K30）",
        "mCSM 保G但中性",
        "PremPRI 优保G(+0.29)",
    ),
    60: (
        "loop3（尾）",
        "可动",
        "0.30",
        "–",
        "mCSM 温和排A(+1.15)",
        "★ PremPRI 判别最强(+0.46)",
    ),
    98: ("WS环", "慎动", "0.91", "–", "mCSM 不可靠(RSA=0)", "–"),
    129: ("loop7（头）", "可动", "0.29", "–", "–", "–"),
    130: ("loop7（探针）", "可动", "0.03", "–", "–", "–"),
    131: ("loop7", "可动", "0.21", "MI 0.87（与K30）", "–", "–"),
    132: (
        "loop7（出口）",
        "可动",
        "0.05",
        "–",
```





```
        "★ mCSM 最高(+9.46)",
        "★ PremPRI 最优(+0.36)",
    ),
    133: ("loop7（尾）", "可动", "0.43", "—", "—", "—"),
}
# 湿最佳（取保留≥0.6 中 ratio 最高；否则取 ratio 最高并标注）
from collections import defaultdict

by_pos = defaultdict(list)
for k, v in wet.items():
    by_pos[v["resi"]].append((k, v["ratio"], v["retain"], v["active"]))
rows2 = []
for resi in sorted(by_pos):
    items = sorted(by_pos[resi], key=lambda x: -x[1])
    ok = [x for x in items if x[3]]
    best = ok[0] if ok else items[0]
    s = SITE.get(resi, ("?", "?", "?", "?", "?", "?"))
    note = "" if ok else "⚠ 该位点无保留≥60%的可用突变"
    rows2.append(
        [
            resi,
            s[0],
            s[1],
            s[2],
            s[3],
            s[4],
            s[5],
            best[0],
            f"{best[1]:.2f}",
            f"{best[2]:.2f}",
            "可用" if best[3] else "不可用",
            note,
        ]
    )
# 湿排名（按最佳可用 ratio）
rows2.sort(key=lambda r: -(float(r[8]) if r[10] == "可用" else 0))
rows2 = [[f"W{i}"] + r for i, r in enumerate(rows2, 1)]

# ---------- Sheet3：预测方法可靠性 ----------
summ = []
# mCSM-NA：与湿实测对照
n = len(rows1)
if n:
    hit5 = sum(1 for r in rows1 if r["mcsm_rank"] <= 5 and r["active"])
    hi5 = sum(1 for r in rows1 if r["mcsm_rank"] <= 5)
    summ.append(
        [
            "mCSM-NA（combined 分）",
            f"{n} 个可比突变",
            f"前 5 名中 {hit5}/{hi5} 个湿实验可用",
            "✗ 不可靠：把 M132D/E 排为并列第 1，但 M132E 实测最弱（1.18）；且对 G27/S98/T31 无法评分",
        ]
    )
if prem:
    pk = [k for k in prem if to_wet_key(k) in wet]
    hi = sorted(pk, key=lambda k: -prem[k])[:5]
    ok = sum(1 for k in hi if wet[to_wet_key(k)]["active"])
    summ.append(
        [
            "PremPRI（score）",
            f"{len(pk)} 个可比突变",
```





```
            f"前 5 名中 {ok}/{len(hi)} 个湿实验可用",
            "✗ 不可靠：把 M132E 排第 2（实测 1.18 最弱），并把 D/E 优劣判反",
        ]
    )
summ.append(
    [
        "共进化 MI（位点对）",
        "351 对",
        "头部对（25–30 = 1.07、30–58 = 0.77）在组合中均被证实有协同",
        "✓ 部分可靠：用于「选哪些位点配」有参考价值，但不预测幅度",
    ]
)
summ.append(
    [
        "CP / ΔCP（突变体重测）",
        "22 个突变体",
        "正确区分 K30H（H29 保G增强）与 M132D/E（ΔCP 不变，出口通道）",
        "✓ 可靠：用于机制判别，不用于排序",
    ]
)
summ.append(
    [
        "Δd_E72 几何指标",
        "22 个突变体",
        "与实测 +1比 回归 r = −0.073（95%CI −0.600~+0.498）",
        "✗ 失败：GCA 侧 AF3 噪声（SD 2.3–3.9 Å）使信度≈0；仅分层后有效（单点 +0.67 / 组合 −0.72）",
    ]
)
summ.append(
    [
        "加性先验模型（MI+空间+单点+保守）",
        "8 组组合",
        "与实测 +1比 r = +0.138",
        "✗ 失败：组合存在显著上位性，线性加和不可外推",
    ]
)
summ.append(
    [
        "相乘模型（est_ratio，1000 组）",
        "1902→1000 组",
        "Top1 G27D+K30H+M132D 未实测；实测最优 27T–30H–58M 在预测中并非首位",
        "✗ 不可靠：仅排序参考，误差 ±30–50%",
    ]
)
summ.append(
    [
        "结构接触（距离/氢键/埋藏）",
        "300 结构",
        "锁定 H29/K30 为 +1 读取器（✓）；但预测 T31 为排A靶点（✗ 实测无效）",
        "◐ 半可靠：定位「谁参与」可靠，判断「改了有没有用」不可靠",
    ]
)
summ.append(
    [
        "全局脱靶 / GCG 靶向比（本轮新增）",
        "117 样本",
        "亲本靶向比 0.93，最佳组合 30H–58M 达 1.87 且保活性 78%（与单位点偏好比一致）",
        "✓ 可靠：两个独立指标（偏好比、靶向比）在头部候选上互相验证",
    ]
)
```





```
# ---------- 输出 ----------
wb = Workbook()
thin = Side(style="thin", color="BBBBBB")
bd = Border(left=thin, right=thin, top=thin, bottom=thin)


def style(ws, widths):
    for j in range(1, ws.max_column + 1):
        c = ws.cell(row=1, column=j)
        c.font = Font(bold=True, color="FFFFFF", size=10)
        c.fill = PatternFill("solid", fgColor=NAVY)
        c.alignment = Alignment(horizontal="center", vertical="center", wrap_text=True)
        c.border = bd
    for i in range(2, ws.max_row + 1):
        for j in range(1, ws.max_column + 1):
            c = ws.cell(row=i, column=j)
            c.font = Font(size=9)
            c.alignment = Alignment(vertical="top", wrap_text=True)
            c.border = bd
            if i % 2 == 0:
                c.fill = PatternFill("solid", fgColor=LIGHT)
    for col, w in zip("ABCDEFGHIJKLM", widths):
        ws.column_dimensions[col].width = w


ws = wb.active
ws.title = "1_突变级对照"
ws.append(
    [
        "干预测排名",
        "突变(干)",
        "突变(湿)",
        "位点",
        "mCSM combined",
        "干序位",
        "PremPRI",
        "湿 +1比",
        "湿 保留率",
        "湿序位",
        "名次差(湿-干)",
        "一致性判定",
    ]
)
ws.append([])
for r in sorted(rows1, key=lambda x: x["mcsm_rank"]):
    ws.append(
        [
            r["mcsm_rank"],
            r["dry"],
            r["wet_key"],
            r["resi"],
            round(r["mcsm"], 2),
            r["mcsm_rank"],
            round(r["prem"], 2) if r["prem"] is not None else "—",
            round(r["ratio"], 2),
            round(r["retain"], 2),
            r["wet_rank"],
            r["rank_diff"],
            r["verdict"],
        ]
    )
```





```
ws.delete_rows(2)
style(ws, [10, 10, 10, 6, 12, 8, 9, 9, 9, 8, 11, 22])

ws2 = wb.create_sheet("2_位点级对照")
ws2.append(
    [
        "湿排名",
        "位点",
        "位点分类",
        "可动性",
        "保守性(WT频率)",
        "共进化 MI",
        "mCSM-NA 预测",
        "PremPRI 预测",
        "湿最佳突变",
        "+1比",
        "保留率",
        "可用性",
        "备注",
    ]
)
for r in rows2:
    ws2.append(r)
style(ws2, [7, 5, 17, 7, 12, 20, 20, 18, 11, 7, 7, 8, 26])

ws3 = wb.create_sheet("3_预测方法可靠性")
ws3.append(["预测手段", "数据量", "与湿实验的一致性", "结论（可靠/不可靠）"])
for r in summ:
    ws3.append(r)
style(ws3, [30, 16, 50, 60])

wb.save("results/干湿排序对照表.xlsx")
print("写入 results/干湿排序对照表.xlsx")

# ---------- 控制台 ----------
print("\n" + "=" * 104)
print("表1  突变级：干预测（mCSM-NA）排名 vs 湿实测（+1比）排名")
print("=" * 104)
print(
    f"{'干排名':>5}{'突变':<7}{'mCSM':>7}{'湿+1比':>8}{'保留':>7}{'湿排名':>7}{'名次差':>7}  判定"
)
for r in sorted(rows1, key=lambda x: x["mcsm_rank"]):
    print(
        f"{r['mcsm_rank']:>5}{r['wet_key']:<7}{r['mcsm']:>7.2f}{r['ratio']:>8.2f}"
        f"{r['retain']:>7.2f}{r['wet_rank']:>7}{r['rank_diff']:>+7}  {r['verdict']}"
    )
if rows1:
    diffs = [abs(r["rank_diff"]) for r in rows1]
    print(
        f"\n平均绝对名次差 = {statistics.mean(diffs):.1f}（随机预期 ≈ {len(rows1)/3:.1f}）"
    )
    sp = sum(1 for r in rows1 if r["verdict"].startswith("✓")) / len(rows1)
    agree = sum(1 for r in rows1 if abs(r["rank_diff"]) <= 2) / len(rows1)
    print(f"名次差 ≤2（大致一致）比例 = {agree:.0%}")
    print("关键错配：")
    for r in sorted(rows1, key=lambda x: -abs(x["rank_diff"]))[:4]:
        print(
            f"  {r['wet_key']}：干第 {r['mcsm_rank']} → 湿第 {r['wet_rank']}"
            f"（+1比 {r['ratio']:.2f} / 保留 {r['retain']:.2f}）"
        )


```





```
print("\n" + "=" * 104)
print("表2  湿实验验证的最佳位点排序（保留≥60% 优先）")
print("=" * 104)
print(
    f"{'排名':>4}{'位点':<5}{'位点分类':<18}{'可动性':<7}{'湿最佳':<7}{'+1比':>7}{'保留':>7}  可用性"
)
for r in rows2:
    print(f"{r[0]:>4}{r[1]:<5}{r[2]:<18}{r[3]:<7}{r[8]:<7}{r[9]:>7}{r[10]:>7}  {r[11]}")

print("\n" + "=" * 104)
print("表3  预测方法可靠性汇总")
print("=" * 104)
for r in summ:
    print(f"\n【{r[0]}】{r[1]}")
    print(f"   一致性：{r[2]}")
    print(f"   结论：{r[3]}")
```





code/historical/code/s7_wetlab/51_metric_reevaluation.py

```python
# -*- coding: utf-8 -*-
"""
阶段 51 —— 用 top-5 / top-10 回收率重新评价此前所有方法

背景（Peng et al. 2026 Mol Cell 的启示）：
  此前一律用 Pearson / Spearman 评价预测方法，结论是「全部失败」。
  但真正决定实用价值的是「能不能把最好的几个排进前几名」——即 top-N 回收率。
  本脚本对每个方法同时给出：Pearson、Spearman、NDCG@10、top-5 回收、top-10 回收，
  并用置换检验给出 top-N 回收率的经验 p 值（小样本下 top-N 的方差很大，必须做检验）。

被评价的方法：
  mCSM-NA      results/mcsm_single_rerank.csv         (score = combined)
  PremPRI      results/prempri_single_rerank.csv      (score = score)
  Δd_E72       results/dE72_regression.csv            (score = delta_d_E72，假设越大越好)
  加性先验模型  results/23_pair_score_matrix.csv       (score = theory)
  CP / ΔCP     results/mut_cp_summary_保G排A.csv       (score = 排A指数 − 保G指数)

产出：results/方法重评价_topN.csv
"""
import csv, sys, math, re, json

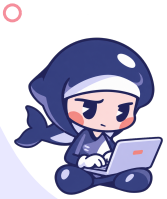
sys.stdout.reconfigure(encoding="utf-8")
import numpy as np


# ---------------------------------------------------------------------------
# 指标
# ---------------------------------------------------------------------------
def pearson(x, y):
    x = np.asarray(x, float)
    y = np.asarray(y, float)
    x = x - x.mean()
    y = y - y.mean()
    d = math.sqrt((x**2).sum() * (y**2).sum())
    return float((x * y).sum() / d) if d > 0 else float("nan")


def spearman(x, y):
    def rk(v):
        s = np.argsort(np.asarray(v, float))
        r = np.empty(len(v))
        r[s] = np.arange(len(v))
        return r

    return pearson(rk(x), rk(y))


def ndcg_at(y_true, y_pred, k):
    yt = np.asarray(y_true, float)
    if np.ptp(yt) <= 0:
        return float("nan")
    yt = (yt - yt.min()) / np.ptp(yt)
    k = min(k, len(yt))
    order = np.argsort(-np.asarray(y_pred, float))[:k]
    ideal = np.argsort(-yt)[:k]
    disc = 1.0 / np.log2(np.arange(2, k + 2))
```





```
    idcg = float((yt[ideal] * disc).sum())
    return float((yt[order] * disc).sum()) / idcg if idcg > 0 else float("nan")


def topn_recovery(y_true, y_pred, n):
    yt = np.asarray(y_true, float)
    n = min(n, len(yt))
    a = set(np.argsort(-np.asarray(y_pred, float))[:n])
    b = set(np.argsort(-yt)[:n])
    return len(a & b) / n


def exact_topn_p(n, k, m):
    """精确超几何 p 值：从 n 个里随机挑 k 个，与真实 top-k 的重叠 >= m 的概率。"""
    from math import comb

    if k > n:
        return float("nan")
    tot = comb(n, k)
    p = sum(
        comb(k, j) * comb(n - k, k - j) for j in range(m, k + 1) if 0 <= k - j <= n - k
    )
    return p / tot


def evaluate(score, outcome, n_perm=5000, seed=0):
    """返回指标字典。top-N 回收率用精确超几何 p 值（置换只能给近似，且小样本下不稳）。
    护栏：n < 10 不报 top-5（组合数太少，无区分力）；n < 15 不报 top-10。"""
    s = np.asarray(score, float)
    o = np.asarray(outcome, float)
    m = np.isfinite(s) & np.isfinite(o)
    s, o = s[m], o[m]
    n = len(s)
    if n < 6:
        return None
    out = dict(
        n=n,
        pearson=pearson(s, o),
        spearman=spearman(s, o),
        ndcg10=ndcg_at(o, s, 10) if n >= 15 else float("nan"),
        top5=float("nan"),
        top5_p=float("nan"),
        top5_note="",
        top10=float("nan"),
        top10_p=float("nan"),
        top10_note="",
    )
    if n >= 10:
        m5 = int(round(topn_recovery(o, s, 5) * 5))
        out.update(top5=m5 / 5, top5_p=exact_topn_p(n, 5, m5))
    else:
        out["top5_note"] = f"n={n}<10，组合数仅 {comb_n(n,5)} 种，无区分力"
    if n >= 15:
        m10 = int(round(topn_recovery(o, s, 10) * 10))
        out.update(top10=m10 / 10, top10_p=exact_topn_p(n, 10, m10))
    else:
        out["top10_note"] = f"n={n}<15，不报 top-10"
    return out


def comb_n(n, k):
```





```
    from math import comb

    return comb(n, k) if k <= n else 0


# ---------------------------------------------------------------------------
# 实测结果
# ---------------------------------------------------------------------------
single = {}
for r in csv.DictReader(open("results/experimental_ranking.csv", encoding="utf-8-sig")):
    try:
        single[r["mut"]] = dict(
            ratio=float(r["ratio"]),
            retain=float(r["retain"]),
            active=r["active"] == "True",
        )
    except ValueError:
        pass
combo = {}
for r in csv.DictReader(open("results/GCG_vs_off_ranking_WT.csv", encoding="utf-8-sig")):
    try:
        tr = float(r["targeting_ratio"])
    except Exception:
        continue
    combo[r["mutation"]] = dict(
        tr=tr, act=float(r["activity_retention"] or 0), finite=not math.isinf(tr)
    )
print(f"实测单点 {len(single)} 个；实测组合 {len(combo)} 个")


def wet_key(name):
    """M132D -> 132D ; G25E -> 25E ; 27T-30H -> 27T-30H ; g27tk30hq58m -> 27T-30H-58M"""
    if not name:
        return None
    s = str(name)
    if s.upper().startswith("WT") or "PUF12-9" in s:
        return None
    if (
        "-" in s
        and re.match(r"^\d", s)
        and len(s.split("-")) > 1
        and not re.match(r"^[A-Z]\d+[A-Z]$", s.split("-")[0])
    ):
        return s  # 已是 27T-30H 形式
    if re.match(r"^[A-Z]\d+[A-Z]$", s):
        m = re.match(r"^([A-Z])(\d+)([A-Z])$", s)
        return f"{m.group(2)}{m.group(3)}"
    # 小写连写形式 g27tk30hq58m
    parts = re.findall(r"([a-z])(\d+)([a-z])", s.lower())
    if parts:
        return (
            "-".join(f"{p}{a.upper()}" for _, p, a in parts).replace("-", "-")
            if False
            else "-".join(f"{int(p)}{a.upper()}" for _, p, a in parts)
        )
    return None


def combo_key(name):
    """把各种组合写法统一成 27T-30H-58M 形式。"""
    if not name:
```





```
        return None
    s = str(name).strip()
    if s.upper().startswith("WT") or "PUF12-9" in s:
        return None
    parts = re.split(r"[+\-]", s)
    out = []
    for p in parts:
        p = p.strip()
        m = re.match(r"^([A-Z])(\d+)([A-Z])$", p)
        if m:
            out.append(f"{m.group(2)}{m.group(3)}")
            continue
        m = re.match(r"^(\d+)([A-Z])$", p)
        if m:
            out.append(f"{m.group(1)}{m.group(2)}")
    return "-".join(out) if out else None


# ------------------------------------------------------------------------
# 逐方法评价
# ------------------------------------------------------------------------
RES = []


def record(
    method, scope, score_name, score, names, keyfn, outcome_name, outcome_map, getter
):
    pts = []
    for nm, sc in zip(names, score):
        k = keyfn(nm)
        if k is None or k not in outcome_map:
            continue
        v = getter(outcome_map[k])
        if v is None:
            continue
        pts.append((float(sc), float(v)))
    if len(pts) < 6:
        print(f"  [跳过] {method} / {outcome_name}：可比样本仅 {len(pts)} 个")
        return
    s, o = zip(*pts)
    e = evaluate(s, o)
    e.update(
        method=method,
        scope=scope,
        score=score_name,
        outcome=outcome_name,
        matched=len(pts),
    )
    RES.append(e)

    def fmt(v, p):
        if not np.isfinite(v):
            return "   —    "
        return (
            f"{v:.2f}" + ("★" if np.isfinite(p) and p <= 0.05 else " ") + f"(p={p:.3f})"
        )

    print(
        f"  {method:<14}{outcome_name:<14}n={e['n']:<3} "
        f"r={e['pearson']:+.3f} ρ={e['spearman']:+.3f}  "
        f"top5={fmt(e['top5'], e['top5_p']):<14}top10={fmt(e['top10'], e['top10_p'])}"
```





```
    )


print("\n" + "=" * 116)
print("一、mCSM-NA（单点，score = combined）")
print("=" * 116)
rows = list(csv.DictReader(open("results/mcsm_single_rerank.csv", encoding="utf-8-sig")))
nm = [r["mut"] for r in rows]
sc = [r["combined"] for r in rows]
record(
    "mCSM-NA",
    "单点",
    "combined",
    sc,
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
record(
    "mCSM-NA",
    "单点",
    "combined",
    sc,
    nm,
    wet_key,
    "活性保留",
    single,
    lambda d: d["retain"],
)
# 子分单独评价：综合分可能把有信号的子分压掉
record(
    "mCSM-swap子分",
    "单点",
    "score_swap",
    [r["score_swap"] for r in rows],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
record(
    "mCSM-off子分",
    "单点",
    "score_off",
    [r["score_off"] for r in rows],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
# 去掉标记 unreliable 的（RSA=0，工具本身说不可信）
rel = [r for r in rows if r["unreliable"] == "False"]
print(f"  去掉 unreliable 后剩 {len(rel)} 个")
record(
    "mCSM-NA(仅可信)",
    "单点",
    "combined",
```





```
    [r["combined"] for r in rel],
    [r["mut"] for r in rel],
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)

print("\n" + "=" * 116)
print("二、PremPRI（单点，score = score）")
print("=" * 116)
rows = list(
    csv.DictReader(open("results/prempri_single_rerank.csv", encoding="utf-8-sig"))
)
nm = [r["mut"] for r in rows]
sc = [r["score"] for r in rows]
record(
    "PremPRI",
    "单点",
    "score",
    sc,
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
record(
    "PremPRI", "单点", "score", sc, nm, wet_key, "活性保留", single, lambda d: d["retain"]
)
record(
    "PremPRI-swap子分",
    "单点",
    "score_swap",
    [r["score_swap"] for r in rows],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
record(
    "PremPRI-off子分",
    "单点",
    "score_off",
    [r["score_off"] for r in rows],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)

print("\n" + "=" * 116)
print("三、Δd_E72（几何指标；假设越大越好）")
print("=" * 116)
rows = list(csv.DictReader(open("results/dE72_regression.csv", encoding="utf-8-sig")))
nm = [r["mut"] for r in rows]
sc = [r["delta_d_E72"] for r in rows]
_d72meas = {
    r["mut"]: float(r["measured_ratio"]) for r in rows if r["measured_ratio"].strip()
```





```
}
print(f"  文件自带配对实测 +1比：{len(_d72meas)} 个")
record(
    "Δd_E72",
    "单点+组合",
    "delta_d_E72",
    sc,
    nm,
    lambda x: x,
    "实测 +1比（配对）",
    _d72meas,
    lambda d: d,
)
record(
    "Δd_E72",
    "单点+组合",
    "delta_d_E72",
    sc,
    nm,
    wet_key,
    "活性保留",
    single,
    lambda d: d["retain"],
)
# 反向，用于核对方向是否判反
record(
    "Δd_E72(反向)",
    "单点+组合",
    "-delta_d_E72",
    [-float(x) for x in sc],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)

print("\n" + "=" * 116)
print("四、加性先验模型（位点对，score = theory）")
print("=" * 116)
rows = list(
    csv.DictReader(open("results/23_pair_score_matrix.csv", encoding="utf-8-sig"))
)
nm = [r["mutA"] + "+" + r["mutB"] for r in rows]
sc = [r["theory"] for r in rows]
meas = {
    combo_key(r["measured_src"]): float(r["measured"])
    for r in rows
    if r.get("measured") and r.get("measured_src")
}
print(f"  表内实测对 {len(meas)} 个")
record("加性先验", "组合", "theory", sc, nm, combo_key, "实测 +1比", meas, lambda d: d)

print("\n" + "=" * 116)
print("五、CP / ΔCP（score = 排A指数 − 保G指数，越大越好）")
print("=" * 116)
rows = list(
    csv.DictReader(open("results/mut_cp_summary_保G排A.csv", encoding="utf-8-sig"))
)
nm, sc = [], []
for r in rows:
```





```
    bg = r["保G指数(H29+K30 ΔCP均值)"]
    pa = r["排A指数(催化核心 ΔCP均值)"]
    nm.append(list(r.values())[0])
    sc.append(float(pa) - float(bg))
record(
    "CP/ΔCP",
    "单点+组合",
    "排A−保G",
    sc,
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)
record(
    "CP/ΔCP",
    "单点+组合",
    "排A−保G",
    sc,
    nm,
    wet_key,
    "活性保留",
    single,
    lambda d: d["retain"],
)
record(
    "CP/ΔCP(反向)",
    "单点+组合",
    "保G−排A",
    [-float(x) for x in sc],
    nm,
    wet_key,
    "+1比（特异性）",
    single,
    lambda d: d["ratio"],
)

# ---------------------------------------------------------------------------
# 汇总表
# ---------------------------------------------------------------------------
print("\n" + "=" * 116)
print("汇总（★ = top-N 回收率置换 p ≤ 0.05）")
print("=" * 116)
print(
    f"{'方法':<16}{'评价对象':<16}{'n':>4}{'Pearson':>9}{'Spearman':>10}"
    f"{'top5':>8}{'p':>8}{'top10':>8}{'p':>8}"
)
for e in RES:
    t5 = f"{e['top5']:.2f}" if np.isfinite(e['top5']) else "—"
    t10 = f"{e['top10']:.2f}" if np.isfinite(e['top10']) else "—"
    p5 = f"{e['top5_p']:.3f}" if np.isfinite(e['top5_p']) else "—"
    p10 = f"{e['top10_p']:.3f}" if np.isfinite(e['top10_p']) else "—"
    print(
        f"{e['method']:<16}{e['outcome']:<16}{e['n']:>4}{e['pearson']:>+9.3f}"
        f"{e['spearman']:>+10.3f}{t5:>8}{p5:>8}{t10:>8}{p10:>8}"
    )

with open("results/方法重评价_topN.csv", "w", newline="", encoding="utf-8-sig") as f:
    w = csv.writer(f)
    w.writerow(
```





```
        [
            "method",
            "scope",
            "score_column",
            "outcome",
            "n_matched",
            "pearson",
            "spearman",
            "ndcg@10",
            "top5_recovery",
            "top5_exact_p",
            "top5_note",
            "top10_recovery",
            "top10_exact_p",
            "top10_note",
        ]
    )
    for e in RES:
        w.writerow(
            [
                e["method"],
                e["scope"],
                e["score"],
                e["outcome"],
                e["n"],
                round(e["pearson"], 4),
                round(e["spearman"], 4),
                round(e["ndcg10"], 4),
                "" if not np.isfinite(e["top5"]) else round(e["top5"], 3),
                "" if not np.isfinite(e["top5_p"]) else round(e["top5_p"], 4),
                e.get("top5_note", ""),
                "" if not np.isfinite(e["top10"]) else round(e["top10"], 3),
                "" if not np.isfinite(e["top10_p"]) else round(e["top10_p"], 4),
                e.get("top10_note", ""),
            ]
        )
print("\n写入 results/方法重评价_topN.csv")
```





code/historical/code/s8_offtarget/58_targeting_ratio_rerank.py

```
# -*- coding: utf-8 -*-
"""
阶段 58 —— 靶向比偏倚的「回补检验」与校正后重新排序

背景（Cycle 3）：
  我们此前发现 targeting_ratio = Σ_GCG / Σ_nonGCG 被 5% 检测下限混淆——
  酶整体活性下降时，脱靶位点率先掉到检出线下被记为 0，分母塌缩、比值虚高。
  当时我们确立了两条规则并**应用**了下限（见 25_offtarget_screen.py:96，
  `if r["on"] < 0.30: continue`），但从未回补检验两件事：
    (1) 加了活性下限之后，偏倚是否真的被消除？
    (2) 若按校正后的指标重新排序，选出的候选会不会与原始排序不同？

本脚本补做这两件事。

【回补检验 Part 1】偏倚是否被下限消除
  比较「全量」与「加下限 C388 ≥ 0.30」两种口径下：
    · Pearson/Spearman(靶向比, Σ非GCG)
    · Pearson(C388%, Σ非GCG%)
    · 低/高活性层靶向比中位数
    · 比值 = ∞ 的构建体数

【回补检验 Part 2】六种排序口径的对比与稳健候选
  (a) 原始 targeting_ratio
  (b) 加活性下限的 targeting_ratio
  (c) 活性残差化（对 C388% 回归取残差）
  (d) 平均非 GCG 编辑率（Σ/n_detected，减轻分母塌缩）
  (e) 归一化脱靶负荷 Σoff/on（25 脚本实际使用的口径）
  (f) +1 比（GCG/GCA，单点对单点，独立于扩增子指标）
  对比各自 top-10 的重叠度，区分「稳健候选」与「原始排序伪影」。

产出：results/靶向比_回补检验.csv
      results/靶向比_重排序对比.csv
"""
import csv, math, sys

sys.stdout.reconfigure(encoding="utf-8")
import numpy as np
from scipy import stats as st

SRC = "results/GCG_vs_off_ranking_WT.csv"
OFF = "results/offtarget_by_mutant.csv"
CMB = "results/23_combo_master_table.csv"
FLOOR = 0.30
OUT1, OUT2 = "results/靶向比_回补检验.csv", "results/靶向比_重排序对比.csv"


def F(x):
    try:
        v = float(x)
        return v if math.isfinite(v) else None
    except (TypeError, ValueError):
        return None


def rd(p):
    with open(p, encoding="utf-8-sig") as f:
```





```
        return list(csv.DictReader(f))


# ---------- 载入 ----------
rows = [r for r in rd(SRC) if r["is_WT"] != "True"]
off = {r["mutant"]: r for r in rd(OFF)}
p1 = {r["muts"]: F(r["ratio_p1"]) for r in rd(CMB)}

for r in rows:
    r["_c388"] = F(r["C388_pct"])
    r["_tr"] = F(r["targeting_ratio"])
    r["_off"] = F(r["sum_nonGCG_pct"])
    r["_ngcg"] = F(r["n_nonGCG_detected"])
    r["_ng"] = F(r["n_GCG_detected"])
    r["_act"] = F(r["activity_retention"])
    o = off.get(r["mutation"], {})
    r["_offsum"] = F(o.get("off_sum"))
    r["_on388"] = F(o.get("on_388"))
    r["_p1"] = p1.get(r["mutation"])

has_off = [r for r in rows if r["_off"] is not None]
fin = [r for r in has_off if r["_tr"] is not None]
keep = [r for r in fin if r["_c388"] is not None and r["_c388"] >= FLOOR * 100]
print(
    f"组合构建体 {len(rows)} 个；有 Σ 非GCG 读数 {len(has_off)}；比值有限 {len(fin)}；"
    f"再加 C388 ≥ {FLOOR*100:.0f}% 后 {len(keep)}"
)

# ============================================================================
# Part 1 —— 偏倚是否被下限消除
# ============================================================================
print("\n" + "=" * 96)
print("Part 1  回补检验：加活性下限后，偏倚被消除了吗？")
print("=" * 96)

res1 = []


def part1(tag, sub):
    off_a = np.array([r["_off"] for r in sub])
    tr_a = np.array([r["_tr"] for r in sub])
    c3_a = np.array([r["_c388"] for r in sub])
    if len(sub) < 5:
        return
    rp = float(np.corrcoef(off_a, tr_a)[0, 1])
    rs = float(st.spearmanr(off_a, tr_a)[0])
    cp = float(np.corrcoef(c3_a, off_a)[0, 1])
    lo = [r["_tr"] for r in sub if 10 <= r["_c388"] <= 20]
    hi = [r["_tr"] for r in sub if 35 <= r["_c388"] <= 60]
    n_inf = sum(
        1
        for r in rows
        if r["_c388"] is not None
        and r["_c388"] >= (FLOOR * 100 if tag.startswith("加下限") else 0)
        and r["_tr"] is None
        and r["_off"] is not None
    )
    med_lo = float(np.median(lo)) if lo else float("nan")
    med_hi = float(np.median(hi)) if hi else float("nan")
    print(f"\n【{tag}】n = {len(sub)}")
    print(f"  Spearman ρ(靶向比, Σ非GCG) = {rs:+.3f}    Pearson r = {rp:+.3f}")
```





```
    print(f"  Pearson  r(C388%, Σ非GCG%)  = {cp:+.3f}")
    print(
        f"  低活性层(10–20%) 中位 {med_lo:.2f} (n={len(lo)})   "
        f"高活性层(35–60%) 中位 {med_hi:.2f} (n={len(hi)})   层间比 {med_lo/med_hi:.2f}×"
    )
    print(f"  该口径下比值 = ∞ 的构建体：{n_inf} 个")
    res1.append(
        dict(
            口径=tag,
            n=len(sub),
            Spearman_靶向比_非GCG=round(rs, 3),
            Pearson_靶向比_非GCG=round(rp, 3),
            Pearson_C388_非GCG=round(cp, 3),
            低活性层中位=round(med_lo, 2),
            低活性层n=len(lo),
            高活性层中位=round(med_hi, 2),
            高活性层n=len(hi),
            层间倍数=round(med_lo / med_hi, 2) if med_hi else None,
            比值无穷数=n_inf,
        )
    )


part1("全量（不过滤）", fin)
part1(f"加下限 C388 ≥ {FLOOR*100:.0f}%", keep)

# ============================================================================
# Part 2 —— 六种口径的重新排序
# ============================================================================
print("\n" + "=" * 96)
print("Part 2  重新排序：六种口径下的 top-10")
print("=" * 96)

sub = [r for r in keep if r["_p1"] is not None or True]
# (c) 活性残差化
c3 = np.array([r["_c388"] for r in keep])
tr = np.array([r["_tr"] for r in keep])
z = np.polyfit(c3, tr, 1)
resid = tr - np.polyval(z, c3)
for r, e in zip(keep, resid):
    r["_resid"] = float(e)


def score(r, key):
    if key == "a":
        return r["_tr"]
    if key == "b":
        return r["_tr"]
    if key == "c":
        return r["_resid"]
    if key == "d":
        n = r["_ngcg"] or 0
        return (r["_off"] / n) if n > 0 else None
    if key == "e":
        o, on = r["_offsum"], r["_on388"]
        return (o / on) if (o is not None and on and on > 0) else None
    if key == "f":
        return r["_p1"]
    return None
```





```
NAMES = {
    "a": "原始 targeting_ratio（全量）",
    "b": f"targeting_ratio + 活性下限",
    "c": "活性残差化比值",
    "d": "平均非GCG编辑率 Σ/n",
    "e": "归一化脱靶负荷 Σoff/on",
    "f": "+1 比（独立口径）",
}
POOL = {"a": fin, "b": keep, "c": keep, "d": keep, "e": keep, "f": keep}

top = {}
for k in "abcdef":
    pool = POOL[k]
    scored = [(r, score(r, k)) for r in pool]
    scored = [(r, s) for r, s in scored if s is not None]
    asc = k in ("d", "e")  # 这两种越低越好
    scored.sort(key=lambda t: t[1], reverse=not asc)
    top[k] = [r["mutation"] for r, _ in scored[:10]]
    print(
        f"\n({k}) {NAMES[k]}  （n = {len(scored)}{'，越低越好' if asc else '，越高越好'}）"
    )
    print("     " + "  ".join(f"{i+1}.{m}" for i, m in enumerate(top[k])))

# 重叠度矩阵
print("\n" + "=" * 96)
print("top-10 重叠度（|A∩B| / 10）")
print("=" * 96)
ks = list("abcdef")
print("        " + "".join(f"{k:>7}" for k in ks))
for a in ks:
    line = f"  ({a})   "
    for b in ks:
        ov = len(set(top[a]) & set(top[b]))
        line += f"{ov:>7}"
    print(line)

# 稳健 vs 伪影
cnt = {}
for k in ks:
    for m in top[k]:
        cnt[m] = cnt.get(m, 0) + 1
robust = sorted([m for m, c in cnt.items() if c >= 4], key=lambda m: -cnt[m])
artifact = [m for m in top["a"] if cnt.get(m, 0) <= 2]
print(f"\n★ 稳健候选（在 ≥4 种口径中都进 top-10，共 {len(robust)} 个）：")
for m in robust:
    r = next(x for x in keep if x["mutation"] == m)
    print(
        f"    {m:<20} 命中 {cnt[m]}/6 种口径   靶向比 {r['_tr']:.2f}   C388 {r['_c388']:.1f}%"
        f"   活性保留 {r['_act']}"
    )
print(
    f"\n⚠ 原始排序的伪影候选（仅在 ≤2 种口径中进 top-10）：{artifact if artifact else '无'}"
)

# 原始排序 top-10 的活性
raw_top = [next(x for x in fin if x["mutation"] == m) for m in top["a"]]
print(f"\n原始 targeting_ratio 排序 top-10 的 on-target 编辑率：")
for r in raw_top:
    print(
        f"    {r['mutation']:<20} C388 = {r['_c388']:>5.2f}%   靶向比 = {r['_tr']:.2f}"
        f"{'   ← 低于活性下限' if r['_c388'] < FLOOR*100 else ''}"
    )
```





```
    )
n_below = sum(1 for r in raw_top if r["_c388"] < FLOOR * 100)
print(f"    → 其中 {n_below}/10 低于活性下限 {FLOOR*100:.0f}%")

# ============================================================================
# Part 3 —— 对最终决策的影响
# ============================================================================
print("\n" + "=" * 96)
print("Part 3  这次偏倚是否影响了我们的最终候选选择？")
print("=" * 96)
import os as _os, re as _re

CANDP = "results/下一步实验突变候选表_61组.csv"
cand = rd(CANDP) if _os.path.exists(CANDP) else []


def canon(s):
    """把 30H-58M / G30H+Q58M / 27T-30H-58M 等写法统一成残基号+氨基酸的有序集合。"""
    toks = _re.findall(r"([A-Z]?)(\d+)([A-Z])", str(s).upper())
    return frozenset(f"{p}{a}" for _, p, a in toks)


cand_sets = {canon(r.get("突变", "")): r for r in cand}

biased_names = {
    m
    for m in top["a"]
    if next(x for x in fin if x["mutation"] == m)["_c388"] < FLOOR * 100
}
hit = sorted(m for m in biased_names if canon(m) in cand_sets)
print(f"  61 条候选表载入：{len(cand)} 条")
print(
    f"  原始排序 top-10 中低于活性下限的构建体（{len(biased_names)} 个）：{sorted(biased_names)}"
)
print(f"  其中进入 61 条候选表的：{hit if hit else '无'}")

# 反向核对：稳健候选有多少进了候选表
rob = [m for m in cnt if cnt[m] >= 4]
rob_in = sorted(m for m in rob if canon(m) in cand_sets)
print(f"\n  8 个稳健候选中进入 61 条候选表的：{rob_in if rob_in else '无'}")
for m in rob:
    r = next(x for x in keep if x["mutation"] == m)
    print(
        f"    {m:<18} {'✓ 已在候选表' if canon(m) in cand_sets else '— 未纳入'}"
        f"   靶向比 {r['_tr']:.2f}  C388 {r['_c388']:.1f}%  活性 {r['_act']}"
    )
print(f"\n  说明：命中者为 C18（G25E+K30H），与 25E-30H 是同一组合；")
print(
    f"        但它在候选表中的入选理由是「复验重复性 + 作为 Q58M 配对（C18/C28）的对照臂」，"
)
print(f"        与 targeting_ratio 排名无关。")
print(f"\n  → 61 条候选表以 +1 比与机制论证为挑选依据，从未使用 targeting_ratio 排序；")
print(f"     本次偏倚未传导到最终候选选择。")
print(f"     但 8 个稳健候选中有 6 个尚未纳入任何一轮实验，可作为补充验证的参考清单。")

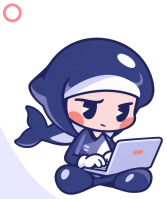
# ---------- 写出 ----------
with open(OUT1, "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=list(res1[0]))
    w.writeheader()
    w.writerows(res1)
print(f"\n写入 {OUT1}")
```





```

rank_rows = []
for m in sorted(cnt, key=lambda x: -cnt[x]):
    r = next((x for x in keep if x["mutation"] == m), None)
    rank_rows.append(
        dict(
            构建体=m,
            命中口径数=cnt[m],
            原始靶向比=round(r["_tr"], 3) if r else "",
            C388_pct=r["_c388"] if r else "",
            活性保留=r["_act"] if r else "",
            活性残差=round(r["_resid"], 3) if r and "_resid" in r else "",
            平均非GCG=round(score(r, "d"), 2) if r and score(r, "d") is not None else "",
            归一化脱靶负荷=(
                round(score(r, "e"), 2) if r and score(r, "e") is not None else ""
            ),
            plus1_ratio=r["_p1"] if r else "",
            判定=("稳健候选" if cnt[m] >= 4 else ("伪影候选" if cnt[m] <= 2 else "中间")),
        )
    )
with open(OUT2, "w", newline="", encoding="utf-8-sig") as f:
    w = csv.DictWriter(f, fieldnames=list(rank_rows[0]))
    w.writeheader()
    w.writerows(rank_rows)
print(f"写入 {OUT2}")
```





scripts/audit_science.py

```python
"""Audit frozen figure inputs and CP clustering without changing scientific settings.

Run from the repository: python scripts/audit_science.py --output work/science-audit.json
Existing output files are never overwritten. No AF3 inference or wet-lab reanalysis.
"""

from pathlib import Path
import argparse, csv, hashlib, json, platform
import numpy as np
import pandas as pd
import scipy
from scipy.spatial.distance import pdist, squareform
from sklearn.cluster import SpectralClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
import sklearn
from threadpoolctl import threadpool_limits

ROOT = Path(__file__).resolve().parents[1]


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument('--output', required=True, type=Path)
    args = ap.parse_args()
    if args.output.exists():
        raise FileExistsError(args.output)
    threadpool_limits(1)
    report = {
        'scope': 'Independent numerical audit of supplied summaries; not experimental validation',
        'versions': {
            'python': platform.python_version(),
            'numpy': np.__version__,
            'pandas': pd.__version__,
            'scipy': scipy.__version__,
            'sklearn': sklearn.__version__,
        },
        'checks': {},
        'input_hashes': {},
    }

    def read(p):
        report['input_hashes'][str(p.relative_to(ROOT))] = hashlib.sha256(
            p.read_bytes()
        ).hexdigest()
        return pd.read_csv(p)

    check = report['checks']
    d = ROOT / 'data/plot-tables'
    single = read(d / 'figure_08_single_site.csv')
    mutants = single[~single.is_parent]
    valid = mutants[mutants.plus1_ratio_defined]
    assert len(mutants) == 102 and len(valid) == 101
    high = valid[valid.activity_retention_vs_same_batch_parent >= 0.70]
    best = high.loc[high.plus1_ratio_GCG_over_GCA.idxmax()]
    assert best.mutant == '30H' and np.isclose(best.plus1_ratio_GCG_over_GCA, 1.71)
    assert not (
```





```
        (valid.plus1_ratio_GCG_over_GCA >= 2)
        & (valid.activity_retention_vs_same_batch_parent >= 0.7)
    ).any()
    check['single_mutants'] = {
        'total': 102,
        'defined': 101,
        'best_at_retention_0_70': 'K30H',
        'ratio': 1.71,
        'retention': 0.84,
    }
    matrix = read(d / 'figure_03_ddcp_matrix.csv')
    full = read(d / 'figure_03_all_residues_ddcp.csv')
    # Input matrix schemas contain two residue annotation columns.
    numeric = full.iloc[:, 2:].to_numpy(float)
    top = np.argsort(-np.nanmax(np.abs(numeric), axis=1))[:22]
    assert set(matrix.iloc[:, 0]) == set(full.iloc[top, 0])
    assert np.isclose(np.nanmax(np.abs(numeric)), 0.395)
    check['ddcp'] = {
        'all_residues': len(full),
        'displayed_residues': len(matrix),
        'constructs': numeric.shape[1],
        'symmetric_limit': 0.395,
    }
    targeting = read(d / 'figure_09_targeting_ratio.csv')
    v = targeting[(~targeting.is_wildtype_parent) & targeting.targeting_ratio_defined]
    assert len(v) == 41
    pearson = v.targeting_ratio.corr(v.sum_nonGCG_editing_pct)
    spearman = v.targeting_ratio.corr(v.sum_nonGCG_editing_pct, method='spearman')
    assert np.isclose(pearson, -0.390128, atol=1e-6) and np.isclose(
        spearman, -0.557017, atol=1e-6
    )
    check['targeting_ratio'] = {
        'defined_mutants': len(v),
        'pearson': pearson,
        'spearman': spearman,
        'interpretation': 'descriptive; ratio and denominator are mathematically coupled',
    }
    candidates = read(d / 'figure_10a_candidate_groups_raw.csv')
    assert candidates.groupby('group').size().to_dict() == {
        'A': 18,
        'B': 6,
        'C': 12,
        'D': 5,
        'E': 20,
    }
    check['candidate_entries'] = 61
    nroot = ROOT / 'figure-sources/orca-n1-n4/input/data'
    combo = read(nroot / 'combo_batch_ratio_vs_retention.csv')
    # The source uses undetected/zero c.871 to flag undefined ratios.
    ratio = 'on_target_GCG_over_offtarget_GCA_ratio'
    ret = 'activity_retention_fraction'
    valid_combo = combo[combo[ratio].notna()]
    assert len(combo) == 48 and len(valid_combo) == 36
    hits = valid_combo[(valid_combo[ratio] >= 2) & (valid_combo[ret] >= 0.7)]
    assert hits.construct.tolist() == ['27T-30H-58M']
    a = valid_combo[[ret, ratio]].to_numpy(float)
    pareto = np.array(
        [not np.any(np.all(a >= p, axis=1) & np.any(a > p, axis=1)) for p in a]
    )
    assert int(pareto.sum()) == 6
    check['combinatorial'] = {
```





```
        'records': 48,
        'defined': 36,
        'pareto': 6,
        'threshold_hit': hits[['construct', ratio, ret]].to_dict('records'),
    }
    designed = read(nroot / 'designed_batch_groups.csv')
    usable = designed[designed.ratio_defined & ~designed.excluded_failed_well]
    assert len(designed) == 37 and len(usable) == 35
    check['designed_batch'] = {
        'records': 37,
        'usable': 35,
        'failed_well_policy': 'inherited from source; lab-level failure has not been independently
established',
    }
    cp = ROOT / 'analysis/cp-clustering'
    zpath = cp / 'variance_cp/results/variance_features.npz'
    report['input_hashes'][str(zpath.relative_to(ROOT))] = hashlib.sha256(
        zpath.read_bytes()
    ).hexdigest()
    z = np.load(zpath, allow_pickle=False)
    x, sx, names = z['construct_features'], z['seed_features'], z['constructs'].tolist()
    assert x.shape == (11, 19110) and sx.shape == (56, 19110)
    assert np.isfinite(x).all() and np.all((x >= 0) & (x <= 1))
    for i, name in enumerate(names):
        assert np.allclose(
            x[i], sx[z['seed_groups'] == name].mean(0), rtol=1e-6, atol=1e-8
        )
    assert int((np.ptp(x, axis=0) > 1e-12).sum()) == 766
    summary = read(cp / 'skill_clustering/results/pipeline_summary.csv')
    labels = read(cp / 'skill_clustering/results/pipeline_labels.csv')
    boot = read(cp / 'skill_clustering/results/pipeline_seed_bootstrap.csv')
    assert len(summary) == 32 and len(boot) == 6400
    max_sil_error = 0.0
    for row in summary.itertuples():
        lab = (
            labels[labels.pipeline == row.pipeline]
            .set_index('construct')
            .loc[names, 'cluster']
            .to_numpy(int)
        )
        mask = lab >= 0
        if 1 < len(set(lab[mask])) < mask.sum():
            score = silhouette_score(x[mask], lab[mask])
            max_sil_error = max(max_sil_error, abs(score - row.common_silhouette))
        b = boot[boot.pipeline == row.pipeline]
        assert len(b) == 200
        assert np.isclose(b.exact_reselected.mean(), row.bootstrap_exact_reselected)
        assert np.isclose(b.exact_fixed_k.mean(), row.bootstrap_exact_fixed_k)
        for r in b.itertuples():
            bl = np.array(json.loads(r.labels))
            assert np.isclose(adjusted_rand_score(lab, bl), r.ARI_reselected)
    assert max_sil_error < 1e-5
    # Independently refit the source-selected spectral baseline. No parameter search is added.
    v = x.var(0, ddof=1)
    v[np.ptp(x, axis=0) <= 1e-12] = 0
    ix = np.lexsort((z['feature_j'], z['feature_i'], -v))[:77]
    a = x[:, ix]
    dist = squareform(pdist(a))
    sigma = np.median(dist[np.triu_indices(11, 1)])
    affinity = np.exp(-(dist**2) / (2 * sigma**2))
    scan = []
```





```
    for k in [2, 3, 4]:
        lab = SpectralClustering(
            n_clusters=k,
            affinity='precomputed',
            assign_labels='cluster_qr',
            random_state=20260915,
        ).fit_predict(affinity)
        scan.append((silhouette_score(a, lab), k, lab))
    best = max(scan, key=lambda q: (q[0], -q[1]))
    frozen = (
        labels[labels.pipeline == 'variable_top10__spectral']
        .set_index('construct')
        .loc[names, 'cluster']
        .to_numpy(int)
    )
    assert best[1] == 3 and adjusted_rand_score(best[2], frozen) == 1
    assert frozen[names.index('t31a')] == frozen[names.index('t31d')]
    check['cp_clustering'] = {
        'constructs': 11,
        'seeds': 56,
        'features': 19110,
        'variable_features': 766,
        'audited_baselines': 32,
        'audited_stored_bootstrap_partitions': 6400,
        'max_common_silhouette_error': max_sil_error,
        'spectral_baseline_refit_ARI': 1.0,
        'selected_k': 3,
        'T31A_T31D_same_cluster': True,
        'not_rerun': 'AF3 inference, 6400 bootstrap fits and UMAP embeddings; stored records audit
.
ed instead',
    }
    report['status'] = 'passed'
    args.output.parent.mkdir(parents=True, exist_ok=True)
    args.output.write_text(json.dumps(report, indent=2, ensure_ascii=False) + '\n')
    print(json.dumps(check, indent=2, ensure_ascii=False))


if __name__ == '__main__':
    main()
```





scripts/compare_esm2.py

```
"""Execute analysis/esm2/PLAN.md with local weights and no network access."""

from pathlib import Path
import argparse, hashlib, importlib.metadata, json, os, platform, re

os.environ['HF_HUB_OFFLINE'] = '1'
os.environ['TRANSFORMERS_OFFLINE'] = '1'
os.environ.setdefault(
    'HF_HOME', str(Path(__file__).resolve().parents[1] / 'work/hf-cache')
)
import numpy as np
import pandas as pd
import torch
from transformers import EsmModel, EsmTokenizer
from scipy.spatial.distance import pdist, squareform
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, adjusted_rand_score
from sklearn.decomposition import PCA
from threadpoolctl import threadpool_limits

ROOT = Path(__file__).resolve().parents[1]


def sha(p):
    return hashlib.sha256(p.read_bytes()).hexdigest()


def cluster(d, k):
    return AgglomerativeClustering(
        n_clusters=k, metric='precomputed', linkage='average'
    ).fit_predict(d)


def main():
    p = argparse.ArgumentParser()
    p.add_argument('--model', required=True, type=Path)
    p.add_argument('--output', required=True, type=Path)
    a = p.parse_args()
    if a.output.exists():
        raise FileExistsError(a.output)
    torch.set_num_threads(2)
    torch.set_num_interop_threads(1)
    threadpool_limits(2)
    torch.manual_seed(20260929)
    np.random.seed(20260929)
    cp = ROOT / 'analysis/cp-clustering'
    manifest = cp / 'variance_cp/results/manifest_with_cluster_comparison.csv'
    wtfile = cp / 'variance_cp/annotation/uploaded_wt.fasta'
    wt = ''.join(
        (l.strip() for l in wtfile.read_text().splitlines() if not l.startswith('>'))
    )
    meta = pd.read_csv(manifest)
    names = meta.construct.tolist()
    seqs = []
    for r in meta.itertuples():
        seq = list(wt)
```





```
        if r.actual_mutations != 'WT':
            for mutation in r.actual_mutations.split(';'):
                old, pos, new = re.fullmatch('([A-Z])(\\d+)([A-Z])', mutation).groups()
                pos = int(pos) - 1
                assert seq[pos] == old, (r.construct, mutation)
                seq[pos] = new
        seqs.append(''.join(seq))
    assert len(set(seqs)) == 11 and all((len(s) == 199 for s in seqs))
    tokenizer = EsmTokenizer.from_pretrained(a.model, local_files_only=True)
    model = (
        EsmModel.from_pretrained(a.model, local_files_only=True, add_pooling_layer=False)
        .float()
        .eval()
    )
    embeddings = {6: [], 12: []}
    with torch.inference_mode():
        for name, seq in zip(names, seqs):
            encoded = tokenizer(seq, return_tensors='pt', return_special_tokens_mask=True)
            residue = (encoded.pop('special_tokens_mask') == 0) & encoded[
                'attention_mask'
            ].bool()
            assert int(residue.sum()) == 199
            result = model(**encoded, output_hidden_states=True)
            for layer in embeddings:
                embeddings[layer].append(
                    result.hidden_states[layer][residue].mean(0).cpu().numpy()
                )
            print('embedded', name, flush=True)
    emb = {f'layer{k}': np.stack(v) for k, v in embeddings.items()}
    source_labels = pd.read_csv(cp / 'skill_clustering/results/pipeline_labels.csv')
    frozen = (
        source_labels[source_labels.pipeline == 'variable_top10__spectral']
        .set_index('construct')
        .loc[names, 'cluster']
        .to_numpy(int)
    )
    dists = {k: squareform(pdist(v, metric='cosine')) for k, v in emb.items()}
    dists['sequence_hamming'] = squareform(
        pdist(np.array([list(s.encode('ascii')) for s in seqs]), metric='hamming')
    )
    a.output.mkdir(parents=True)
    rows = []
    scans = []
    loco = []
    labs = {}
    for label, d in dists.items():
        d = np.maximum(d, 0)
        np.fill_diagonal(d, 0)
        assert np.isfinite(d).all() and np.allclose(d, d.T)
        candidates = []
        for k in [2, 3, 4]:
            c = cluster(d, k)
            sil = silhouette_score(d, c, metric='precomputed')
            candidates.append((sil, k, c))
            scans.append(
                {
                    'representation': label,
                    'k': k,
                    'full_space_silhouette': sil,
                    'cluster_sizes': json.dumps(np.bincount(c).tolist()),
                }
```





```
            )
        sil, k, c = max(candidates, key=lambda z: (z[0], -z[1]))
        labs[label] = c
        aris = []
        for omit, name in enumerate(names):
            keep = np.arange(len(names)) != omit
            sub = cluster(d[np.ix_(keep, keep)], k)
            ari = adjusted_rand_score(c[keep], sub)
            aris.append(ari)
            loco.append(
                {
                    'representation': label,
                    'omitted': name,
                    'fixed_k': k,
                    'ARI_to_restricted_baseline': ari,
                }
            )
        rows.append(
            {
                'representation': label,
                'selected_k': k,
                'full_space_silhouette': sil,
                'cluster_sizes': json.dumps(np.bincount(c).tolist()),
                'singletons': int((np.bincount(c) == 1).sum()),
                'ARI_to_CP_spectral': adjusted_rand_score(frozen, c),
                'T31A_T31D_same_cluster': bool(
                    c[names.index('t31a')] == c[names.index('t31d')]
                ),
                'LOCO_mean_ARI': float(np.mean(aris)),
                'LOCO_min_ARI': min(aris),
                'LOCO_exact_count': int(np.isclose(aris, 1).sum()),
                'LOCO_runs': 11,
            }
        )
        pd.DataFrame(d, index=names, columns=names).to_csv(
            a.output / f'{label}_distances.csv'
        )
    pd.DataFrame(rows).to_csv(a.output / 'summary.csv', index=False)
    pd.DataFrame(scans).to_csv(a.output / 'k_scan.csv', index=False)
    pd.DataFrame(loco).to_csv(a.output / 'leave_one_out.csv', index=False)
    labtable = pd.DataFrame(
        {
            'construct': names,
            'observed_name': meta.observed_display_name,
            'CP_spectral': frozen,
            **labs,
        }
    )
    labtable.to_csv(a.output / 'labels.csv', index=False)
    pca = PCA(n_components=2).fit(emb['layer12'])
    xy = pca.transform(emb['layer12'])
    pd.DataFrame({'construct': names, 'PC1': xy[:, 0], 'PC2': xy[:, 1]}).to_csv(
        a.output / 'pca_display.csv', index=False
    )
    np.savez_compressed(a.output / 'embeddings.npz', constructs=np.array(names), **emb)
    (a.output / 'sequences.fasta').write_text(
        ''.join((f'>{n}\n{s}\n' for n, s in zip(names, seqs)))
    )
    versions = {
        x: importlib.metadata.version(x)
        for x in [
```





```
            'torch',
            'transformers',
            'tokenizers',
            'safetensors',
            'numpy',
            'pandas',
            'scipy',
            'scikit-learn',
        ]
    }
    provenance = {
        'model': 'facebook/esm2_t12_35M_UR50D',
        'device': 'CPU',
        'threads': 2,
        'python': platform.python_version(),
        'versions': versions,
        'input_sha256': {
            'manifest': sha(manifest),
            'wt_fasta': sha(wtfile),
            'plan': sha(ROOT / 'analysis/esm2/PLAN.md'),
        },
        'model_sha256': {
            p.name: sha(p)
            for p in a.model.iterdir()
            if p.is_file()
            and p.name
            in ['config.json', 'model.safetensors', 'vocab.txt', 'tokenizer_config.json']
        },
        'pooling': 'mean of 199 residue embeddings; special tokens excluded',
        'layers': [6, 12],
        'network': 'disabled during inference',
        'activity_labels_used_for_fitting': False,
        'PCA_display_explained_variance': pca.explained_variance_ratio_.tolist(),
        'primary_vs_layer6_ARI': adjusted_rand_score(labs['layer12'], labs['layer6']),
        'primary_vs_sequence_hamming_ARI': adjusted_rand_score(
            labs['layer12'], labs['sequence_hamming']
        ),
        'limitations': [
            '11 near-identical engineered sequences',
            'no held-out functional validation',
            'LOCO sensitivity is not experimental replication',
        ],
    }
    (a.output / 'provenance.json').write_text(json.dumps(provenance, indent=2) + '\n')
    print(pd.DataFrame(rows).to_string(index=False))


if __name__ == '__main__':
    main()
```





requirements-audit.txt

```
# Versions used for the executed lightweight audit; use an isolated environment.
numpy==1.26.4
pandas==2.2.3
scipy==1.13.1
scikit-learn==1.5.2
threadpoolctl==3.6.0
```





code/historical/requirements.txt

```
# APOBEC 编辑模块 · 计算工作流 依赖清单
# 安装：uv pip install -r requirements.txt  或  pip install -r requirements.txt
# 说明：多数分析脚本只用标准库，下列依赖按阶段可选。
#
# ── 核心（多数分析/绘图脚本需要）──
numpy
scipy
matplotlib
openpyxl
#
# ── 文档与演示生成 ──
python-docx
python-pptx
#
# ── 按需（仅个别脚本使用）──
gemmi
pydssp
pysam
scikit-learn
fonttools  # plot_drylab_figures.py：可变字体实例化
#
# 注意：pptx_build.py 使用 PowerPoint COM（仅 Windows，需已安装 Office）。
```



